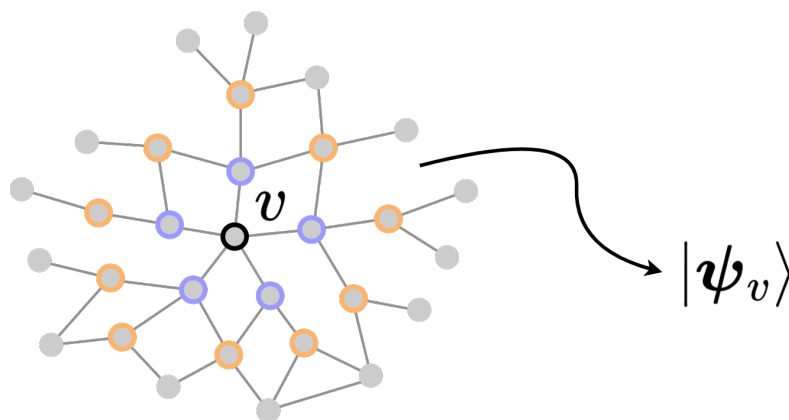




Quantum Computation for Cancer Driver Gene Prediction using Genomic Network Data

Patrícia Simões Marques

Thesis to obtain the Master of Science Degree in

Data Science and Engineering

Supervisors: Dr. Bruno Gabriel Coelho Coutinho
Dr. Andreas Miroslaus Wichert

Examination Committee

Chairperson: Prof. Name of the Chairperson
Supervisor: Dr. Bruno Gabriel Coelho Coutinho
Members of the Committee: Prof. Name of First Committee Member
Dr. Name of Second Committee Member
Eng. Name of Third Committee Member

November 2025

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you – Thank you.

Abstract

Identification of cancer driver genes is fundamental for developing targeted therapeutic interventions. Integrating mutational profiles with protein-protein interaction (PPI) networks provides a systematic approach for their detection, though scaling to large networks poses significant computational challenges. In this thesis, we introduce a supervised quantum framework for cancer driver gene prediction, incorporating the novel *Quantum Multi-order Moment Embedding* (QMME) protocol in the state preparation step. QMME encodes low-order statistical moments of a node's immediate and second-order neighbors (with each node representing a gene) into the amplitudes of quantum states. These amplitude-encoded states serve as inputs to a kernel-based quantum classifier. The pipeline operates almost entirely within the quantum domain from state preparation to prediction, with only minimal classical post-processing inherent to the quantum classifier. Complexity analysis shows that the protocol ensures polylogarithmic qubit resources with constant-depth circuits, offering favorable scaling compared with classical methods. Numerical simulations on an empirical PPI network demonstrate competitive performance, achieving a 12.6% recall gain over a classical baseline and highlighting QMME's capacity to identify rare cancer driver genes. This work establishes a proof-of-principle for an almost fully quantum binary classification pipeline for genomic network data and remains applicable to any feature-labeled graph with partially labeled nodes, providing a foundation for scalable quantum models in complex network inference.

Keywords

Cancer Genomics, Protein-Protein Interaction Networks, Quantum Graph Algorithms, Quantum State Preparation, Quantum Machine Learning

Resumo

Identificação dos genes responsáveis pelo cancro é crucial para desenvolver intervenções terapêuticas. Integrar perfis mutacionais com redes de interação proteína-proteína (IPP) fornece uma abordagem sistemática para a sua deteção, embora apresente desafios computacionais em redes de grande dimensão. Nesta tese, introduzimos um esquema de aprendizagem supervisionada para a previsão de genes responsáveis pelo cancro, incorporando o novo protocolo *Quantum Multi-order Moment Embedding* (QMME) na etapa de preparação do estado quântico. O QMME codifica os momentos estatísticos de baixa ordem dos vizinhos imediatos e de segunda ordem de um nodo (que representa um gene) nas amplitudes de estados quânticos. Estes servem como entradas para um classificador quântico baseado em similaridades. O algoritmo opera quase inteiramente no domínio quântico, desde a preparação até a previsão, apenas com pós-processamento clássico mínimo, inerente ao classificador. A análise de complexidade indica que o protocolo requer um número polilogarítmico de qubits e circuitos de profundidade constante, garantindo escalabilidade favorável. Simulações numéricas numa rede IPP empírica revelam desempenho competitivo, com um aumento de 12,6% na tese de verdadeiros positivos em relação ao método clássico, evidenciando a capacidade do algoritmo para identificar genes raros associados ao cancro. Este trabalho estabelece uma prova de conceito para um modelo de classificação binária quase inteiramente quântico para dados de redes genómicas e mantém-se aplicável a qualquer grafo cujos nodos possuam atributos e estejam parcialmente rotulados, fornecendo uma base para modelos quânticos escaláveis em situações de inferência em redes complexas.

Palavras Chave

Genómica do Cancro, Redes de Interação Proteína-Proteína, Algoritmos Quânticos em Grafos, Preparação de Estados Quânticos, Aprendizagem Quântica

Contents

1	Introduction	1
1.1	Overview	3
1.2	State of the Art	4
1.3	Objectives	4
1.4	Outline	4
1.5	Original Contributions	4
2	Background	7
2.1	Cancer Genomics	9
2.2	Networks in Cancer Biology	11
2.2.1	Graph Theory	11
2.2.2	Protein-Protein Interaction Networks	11
2.2.3	Network-based Feature Extraction	13
2.3	Quantum Computing	13
2.3.1	Qubits and Quantum Operations	14
2.3.2	Gate-Based Quantum Computing	15
2.3.3	Block-Encodings	16
2.3.4	Hybrid Computing	18
2.3.4.A	Implicit Approach (Kernel Estimator + Classical SVM)	19
2.3.4.B	Explicit Approach (Variational/Parameterized Quantum Circuits)	21
3	Quantum Embedding Protocol	25
3.1	Introduction	27
3.1.1	Notation	27
3.1.2	Node Embeddings	27
3.1.3	Problem Statement	29
3.2	Input Model	30
3.3	Quantum Multi-order Moment Embedding (QMME) Algorithm	32
3.3.1	Rotation Operators	32

3.3.2	Feature and Degree-Inverse Oracles	33
3.3.3	Embedding Circuit Design	34
3.4	Complexity Analysis	37
4	Quantum Classification Protocol	41
4.1	Introduction	43
4.2	Quantum Classifier Design	44
4.3	Connection to Classical Models	45
4.4	Complexity Analysis	47
5	Simulation and Implementation	49
5.1	Graph Augmentation	51
5.2	Synthetic Dataset/Toy example	52
5.3	Real Network Data	52
5.3.1	Dataset Description	52
5.3.2	Performance Evaluation	53
5.4	Complexity Analysis	54
6	Concluding Remarks	57
6.1	Conclusions	59
6.2	Limitations and Future Work	59
	Bibliography	61
A	Code of Project	69
B	Drafts and A Large Table	71
B.1	Classification Unitary	71
B.1.1	HHL-inspired linear regression/classification (Amplitude-Encoding the Class) . . .	72
B.2	Quantum Node Embedding Step-by-step Derivation	74
B.3	Complexity Analysis in Detail	76

List of Figures

1.1	Cancer driver genes prediction. (a) PPI network where genes are colored by class. (b) Novel quantum approach: QMME encodes network-based features into quantum states and applies the swap-test classifier. (c) Existing classical approaches rely on tabular features with standard classifiers.	3
2.1	Transcription and translation. (a) https://www.cancer.gov/publications/dictionaries/cancer-terms/def/transcription	10
2.2	Bloch sphere (images side-by-side)	15
3.1	Circuit of the feature oracle O_X, built from controlled rotations F_X. (a) Gate representation of O_X . (b) Detailed view: a two-qubit control register $ i\rangle$ determines how many of the four ancilla qubits undergo F_X rotations ($00 \rightarrow$ first qubit, $01 \rightarrow$ first two, $10 \rightarrow$ first three, $11 \rightarrow$ all four). A d' -qubit register is used internally but omitted for clarity.	33
3.2	Quantum Multi-order Moment Embedding (QMME) protocol. (a) Gate view of the unitary operator U_G , which maps $ v\rangle 0\rangle^{\otimes m} \mapsto v\rangle \psi_v\rangle$, amplitude-encoding low-order statistical moments of node v 's neighborhood. Here, $ v\rangle$ encodes the node index $v \in V$, and the m -qubit data register is initialized to zero. (b) Input PPI network $\mathcal{G} = (V, E, x)$ with node features x representing mutation scores. Immediate neighbors of v are shown in purple, second-hop neighbors in orange. These provide the data for the embedding $ \psi_v\rangle \in \mathbb{R}^{2^m}$. (c) The node embedding function m_v , based on the moments $\varphi(\cdot)$ over v 's neighborhood, is encoded as the quantum state $ \psi_v\rangle$. (d) Circuit-level implementation of the gate U_G . The m -qubit register is composed of subregisters for immediate and second-hop (ℓ_1, ℓ_2) neighbors, a control qubit c , a two-qubit register i indexing the i -th moment, and a five-qubit ancilla register a . The circuit accesses the PPI network in (b) via the quantum oracles in eq. (3.18).	37

4.1	Full cancer gene prediction pipeline with the QMME protocol as part of state preparation. The first register corresponds to the ancilla qubit a , the second to the test data qubits d_t , the third to the index qubits of the test node v_t , the fourth stores the label qubit y , the fifth the labeled data qubits d_l , and the sixth the index qubits of the labeled nodes v_l . The operator U_G , together with the Hadamard gate and the oracle O_Y , prepares the quantum state, which is then processed by the classifier $\mathcal{C}$, i.e., the swap-test classifier. Node labels $\hat{y}_v \in \{0, 1\}$ are predicted via a swap-test and two-qubit measurement statistics on the ancilla and label registers. An outcome of label one indicates that the test gene is predicted as a cancer driver.	46
5.1	Experimental setup for QMME and classical embedding comparison and performance evaluation. (a) The dataset consists of 6 850 genes, of which 590 (in orange) are known cancer driver genes [1]. The remaining 6 260 genes have unknown labels, as they are not part of the CGC dataset. (b) Embeddings for each gene v are obtained via two methods: the Quantum Multi-order Moment Embedding (QMME) protocol, and the Moment Propagation (MoPro) embedding with hyperparameter $t = 1$ [2]. The QMME approach is implemented via algebraic computations on classical software. (c) In <i>each random split</i> , 80% of the positive genes are selected and combined with an equal number of unlabeled genes (assigned a negative class label) to form the labeled training set V_l . The remaining genes form the test set V_t , whose labels are used only for evaluation. For both embedding methods, kernel matrices are computed and the expectation values in eq. (4.9) are evaluated on all test genes. This process is repeated across 50 random splits, and performance metrics are averaged.	53

List of Tables

2.1	Comparison of relevant works and their connection to our quantum embedding method	13
2.2	A nice Spreadsheet using package “spreadtab”. Notice the calculations.	16
3.1	Quantum circuit registers for Quantum Multi-order Moments Embedding (QMME).	36
5.1	Cancer gene classification performance (%) . Comparison of Quantum Multi-order Moment Embeddings (QMME) and Moment Propagation (MoPro) embeddings over 50 random splits. All metric differences are statistically significant except AUPRC.	54
5.2	Complexity analysis for the QMME protocol. U_G complexity estimates for s -sparse and general graphs, in terms of gate count, circuit depth, qubit requirements, and query complexity (calls to the input model).	55
B.1	Example table	73
B.2	Comparison between today’s and target Architectures of Telcos	73
B.3	Example of a very long table spreading in several pages	74
B.4	Sample Table.	74
B.5	Quantum gate complexity and queries for various sources.	77

Acronyms

DNA	Deoxyribonucleic Acid
IQP	Instantaneous Quantum Polynomial Time
LSP	Linear System Problem
PQC	Parameterized Quantum Circuit
QC	Quantum Classification
QCP	Quantum Classification Protocol
QMME	Quantum Multi-order Moments Embedding
QMMEP	Quantum Multi-order Moments Embedding Protocol
RNA	Ribonucleic Acid

1

Introduction

Contents

1.1 Overview	3
1.2 State of the Art	4
1.3 Objectives	4
1.4 Outline	4
1.5 Original Contributions	4

1.1 Overview

Quantum computing is getting real [3]. Quantum algorithms promise quantum advantages in certain computational tasks over existing classical methods. Nevertheless, demonstration of the quantum advantage in practical and industrial applications remains on the agenda [4]. A discipline for which quantum techniques are expected to be beneficial is machine learning [4]. Node-level tasks are concerned with predicting the identity or role of each node within a graph. This project involves using quantum computing to predict data in network nodes, focusing on protein interactions and cancer. The protocol has these main steps: Analyzing moments and how information spreads, Using quantum machine learning to calculate the likelihood of cancer. Key parts of the project: Learning quantum computing in depth, Using a quantum method called QSVD (Quantum Singular Value Decomposer), Figuring out how complex and costly the protocol is to run, Working with SSH for secure connections. Quantum computing sits at the intersection of computer science, mathematics, and physics. While its theoretical foundations - such as quantum algorithms and complexity - are largely mathematical, practical implementations rely on the principles of quantum physics. This thesis focuses on using quantum computing to better understand how proteins interact with cancer and potentially predict cancer risks.

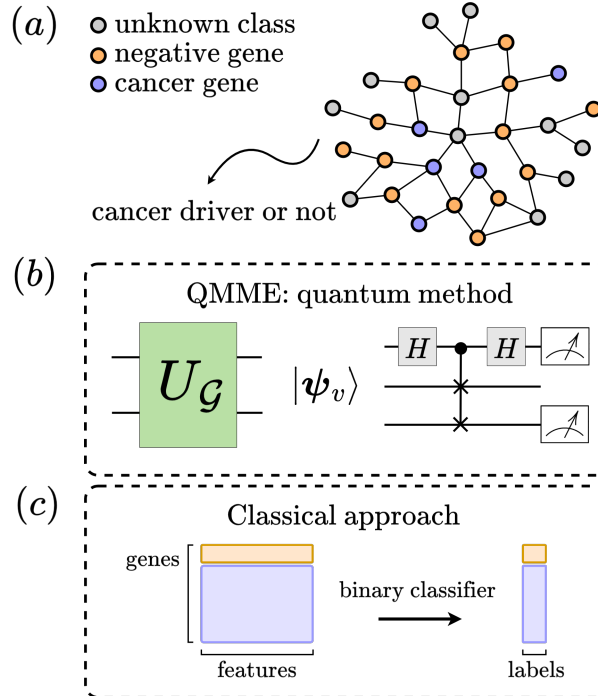


Figure 1.1: Cancer driver genes prediction. (a) PPI network where genes are colored by class. (b) Novel quantum approach: QMME encodes network-based features into quantum states and applies the swap-test classifier. (c) Existing classical approaches rely on tabular features with standard classifiers.

1.2 State of the Art

1.3 Objectives

In this work, we aim to develop a quantum algorithm for cancer gene prediction, including the design of quantum circuits, methodological development, and evaluation of limitations. The focus is on supervised binary classification over biological networks, with the objective of predicting whether a given gene is a cancer driver based on network connectivity and gene-specific features. Our approach integrates a quantum node embedding unitary operator, derived from the node features and graph structure, with a quantum binary classifier to perform node-level predictions. This requires the explicit construction of the state preparation step and the full quantum circuit for classification. The algorithmic efficiency will be analyzed in terms of query complexity, gate complexity, and classical versus quantum performance. Simulations will be performed using platforms such as the IBM Quantum Information Science Kit (Qiskit), and validation will be carried out on both synthetic and real biological datasets, i.e., empirical protein-protein interaction datasets.

1.4 Outline

This thesis is organized in the following way: in Chapter 2, we provide the background necessary to give context for the rest of the work. We start with the relevant concepts of cancer genomics in Section 2.1, followed by a brief introduction on complex networks in Section 2.2 and how they relate to cancer driver gene discovery. In Section 2.3, we cover the fundamentals of quantum computing, describing qubits and basic quantum operations. Section 2.3.4 introduces hybrid computing approaches. In Chapter 3, we introduce the algorithm created for state preparation. Moving to chapter 4, we introduce the quantum classification framework. In Chapter 5, we implement the proposed classification pipeline on empirical data, provide results on performance evaluation, and a brief complexity analysis of the whole algorithm. Finally, Chapter 6 presents concluding remarks and discusses possible future work.

1.5 Original Contributions

The primary contribution of this thesis is the design and validation of an almost fully quantum pipeline for cancer driver gene prediction, evaluated on an empirical protein-protein interaction dataset. In particular, it introduces an explicit state preparation scheme that encodes network-based statistical moments of a node's neighbors' features into the amplitudes of quantum states. The resulting work has been set to be published as: P. Marques, A. Wichert, and B. Coutinho, "Quantum Network-Based Prediction of Cancer

Driver Genes.”

2

Background

Contents

2.1 Cancer Genomics	9
2.2 Networks in Cancer Biology	11
2.3 Quantum Computing	13

2.1 Cancer Genomics

A central objective in oncology is to identify and catalogue genes underlying human cancer [5–7]. While genes with high mutation rates are likely drivers, most *cancer driver genes* have few mutations. In these cases, frequency-based methods alone cannot reliably distinguish rare cancer driver genes from irrelevant ones [6]. Deoxyribonucleic Acid (DNA) is the molecule that stores hereditary information, while Ribonucleic Acid (RNA) serves as its messenger and functional counterpart, carrying instructions from DNA to synthesize proteins and, in some cases, directly regulating gene expression. Cancer driver genes are genes whose alterations (mutations, copy number changes, or fusions) give cells a growth or survival advantage, contributing directly to cancer development. Genes are specific sequences of DNA letters that code for proteins, which are the molecules responsible for carrying out nearly all cellular functions. The process occurs in two stages: transcription, where the DNA sequence is copied into messenger RNA (mRNA), and translation, where ribosomes decode the mRNA into a chain of amino acids that folds into a functional protein. One explanation for why frequency-based approaches fall short lies in the networked nature of cellular processes: genes operate within pathways and complexes, so cancer may result from disruption at the pathway level rather than mutations in specific individual genes. Recent research has embraced this interaction-based perspective of cancer, integrating biological networks with statistical measures of gene-cancer associations to identify novel candidate cancer driver genes [8–11]. (More on Section 2.2.2). Genes act in concert within biological networks, particularly through the proteins they encode. Proteins are long chains of amino acids that carry out a wide range of cellular functions. A special class of proteins, known as enzymes, act as biological catalysts by accelerating biochemical reactions. The three-dimensional structure of proteins determines how they interact, whether briefly, as in the brief encounter between an enzyme and its substrate, or stably, as in structural protein complexes. Through these interactions, proteins may modify one another, assemble into larger complexes, or regulate gene expression by interacting directly with DNA or RNA. The ability of proteins to interact depends on the complementarity of their surface shapes, with specific regions of each protein responsible for binding. Proteins are produced via transcription and translation: the DNA sequence is first transcribed into RNA, which is then translated into a chain of amino acids that folds into a functional protein. These proteins interact with each other to form pathways, complexes, and networks, orchestrating cellular processes. DNA: molecule that carries genetic information. DNA lives in the nucleus of most cells, tightly packed into chromosomes. There's also a little DNA in the mitochondria. Each gene is made up of a sequence of DNA letters. Mutations are changes in the DNA sequence that can alter the function of the encoded protein. Such mutations may disrupt transcription, alter RNA splicing, or change the amino acid sequence of a protein, potentially impairing its function or creating aberrant activities that drive cancer. Some mutations confer a growth or survival advantage to cells and are referred to as cancer driver mutations. The cancer genome is the complete set of DNA, including all the

mutations, found in the cells of a specific cancer. Mutated DNA in cancer cells from many patients. The tumor exome refers to the collection of all exons (the coding regions of genes) from the DNA of a tumor. Large-scale cancer genome projects compile mutated DNA across many patients' cancer cells, such as The Cancer Genome Atlas Program (TCGA) [12, 13]. The cancer genome refers to the complete DNA content of cancer cells, including all somatic mutations. The tumor exome, by contrast, includes only the coding regions of these genes. Large-scale projects, such as The Cancer Genome Atlas (TCGA), systematically catalog mutations across thousands of patient tumors, enabling the study of patterns and identification of candidate cancer driver genes. Because only exons are transcribed into RNA and ultimately translated into proteins, tumor exome sequencing provides a focused view of mutations that are most likely to alter cellular function. The advancement of DNA sequencing technologies now enables the processing of thousands of tumours of many types for systematic mutation discovery. This expansion of scope, coupled with appreciable progress in algorithms [14–18], has led directly to characterization of significant functional mutations, genes and pathways [19–30]. Cancer encompasses more than 100 related diseases [31], making it crucial to understand the commonalities and differences among various types and subtypes. TCGA was founded to address these needs, and its large data sets are providing unprecedented opportunities for systematic, integrated analysis. (From [13]). Because genes and proteins function within interconnected pathways, examining mutation frequency alone may miss important contributors to cancer. Understanding the networked interactions of proteins provides a more complete view, naturally leading to the study of protein-protein interaction networks, as discussed in Section 2.2.

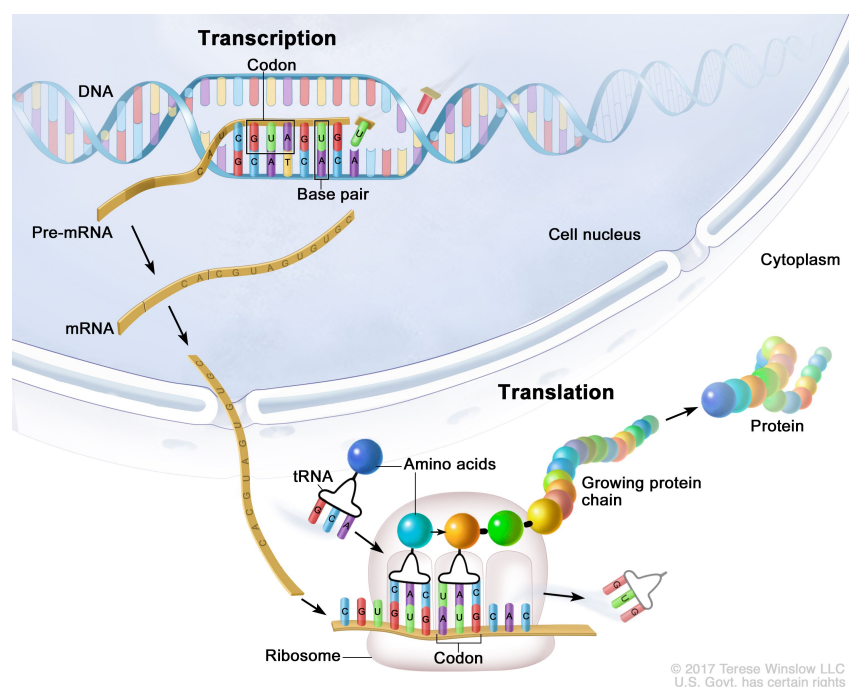


Figure 2.1: Transcription and translation. (a)

<https://www.cancer.gov/publications/dictionaries/cancer-terms/def/transcription>.

2.2 Networks in Cancer Biology

2.2.1 Graph Theory

A network is a collection of interacting elements, typically represented as a graph. The human interactome — a comprehensive network of human protein-protein interactions (PPI) within the human cell (be concrete: InWeb?) — serves as a valuable resource for discovering new cancer-related genes and potential drug combinations. As the complexity of biological networks continues to grow, developing innovative algorithms for efficient network analysis becomes increasingly important.

Theorem 2.2.1. Block-Encodings *Let $G = (V, E)$ be a graph and $(U_1, T_1), (U_2, T_2), \dots, (U_k, T_k)$, be matrix. Then among all forests containing $T = \cup^k j = iT_j$, there is an optimal one block-encoding (u, v) .*

One elegant and memory-efficient way of representing sparse matrices is as adjacency lists.

2.2.2 Protein-Protein Interaction Networks

Gene's PPI network: The set of proteins that are known to physically or functionally interact with the protein encoded by that gene. In these networks, nodes correspond to genes and edges represent relationships between them. Protein-protein interaction (PPI) networks [32–34] are a widely used representation of gene interactions, as they integrate information from diverse data sources, tissues, and molecular processes. Previous studies have combined PPI networks with gene-level cancer association scores, using clustering or network propagation methods. - Network-based discovery: rather than studying genes in isolation, look at how genes interact with each other in biological networks (like human protein-protein interaction networks) - be concrete, is it InWeb?. Instead of asking, “Is this one gene mutated a lot?”*

Protein–protein interaction (PPI) networks can be understood as consisting of several layers. At the most direct level, proteins may physically bind to one another to form complexes. Interactions can also be functional, meaning that proteins are involved in the same biological process, share similar outcomes, or influence each other indirectly through intermediate molecules. Another important layer occurs when one protein regulates the activity of another through post-translational modifications—chemical changes made to a protein after it has been synthesized by ribosomes during translation. Ribosomes themselves are large complexes made up of ribosomal proteins and ribosomal RNA (rRNA), and they serve as the site of protein synthesis in the cell, linking amino acids together into chains. Finally, PPI networks are often context-specific, differing according to disease state, tissue type, or developmental stage. This contextual variation can be exploited to identify therapeutic targets, since proteins may interact differently depending on the biological environment.

We consider a PPI network with N nodes or vertices, with $N = 2^n$ where n is an integer. The adjacency matrix is s -sparse and is represented by A , with $s = 2^5$ and $s \ll n$. In this context, $s = k_{\max}$, the maximum degree of the network.

There exists a feature value for each vertex $v \in V$, denoted by $x_v \in \mathbb{R}$ or in vector notation by $\vec{x} \in \mathbb{R}^N$. We assume this feature value is normalized such that $x_v \in [0, 1]$ for all vertices $v \in V$. In vector notation, this corresponds to $\vec{x} \in [0, 1]^N$. This feature value can be... A common choice for these gene scores is the MutSig P -value [35, 36], which estimates the significance of observed mutations by considering (i) the overall mutational burden, (ii) clustering of mutations within a gene's coding sequence, and (iii) the functional impact of mutations.

NetSig, for instance, focuses on the local neighborhood of each gene to identify cancer driver genes while correcting for node degree bias [8]. - NetSig: Network Significance NetSig asks, "Are this gene's neighbors in the network showing signs of being involved in cancer?" Detect important but rarely mutated genes based on their context. Recent network-based methods like NetSig (Horn et al., 2018) detect cancer driver genes by assessing mutation patterns within a gene's local network neighborhood, without prior knowledge of gene function. Although effective, NetSig operates in an unsupervised manner, relying on known cancer genes mainly for validation after the fact. "An important feature of NetSig is that it is explicitly designed to disregard any mutation information on the gene being tested so that the signal comes from the genes network alone. This ensures that NetSig P values are fully independent of those from existing gene-based statistical test such as MutSig, Oncodrive, GISTIC and RAE (in fact, the MutSig and NetSig P values of the same genes are only modestly correlated, Pearson correlation coefficient = 0.05, data not shown)."

A few studies have explored supervised approaches, incorporating existing knowledge of known driver genes during prediction. Gumpinger et al. used Cancer Gene Census (CGC) annotations to train classifiers for discovering new cancer driver genes [2]. This method models the task as a node-level supervised classification problem in a molecular interaction network, combining network topology with mutation-score features. It achieved more than a 10% improvement in area under the precision–recall curve (AUPRC) over baseline methods. In contrast, the 2021 study Prediction of Cancer Driver Genes through Network-Based Moment Propagation introduced a supervised framework that leverages labeled cancer genes to propagate local statistical features—such as mean and variance of mutation scores—across protein interaction networks for improved gene classification.

The k -hop neighborhood of a vertex $v \in V$ is the set of all genes that can be reached from v along at least k edges, denoted by $\mathcal{N}_v^k$. For a vertex v , we denote with $\mathcal{X}_v^k = \{x_u | u \in \mathcal{N}_v^k\}$ the feature values of vertices in $\mathcal{N}_v^k$. We denote by $\mathcal{X}^k$ the collection of sets $\mathcal{X}_v^k$ for all $v \in V$. [?]. Table B.2 is automatically compressed to fit text width. You can use <https://www.tablesgenerator.com> to produce these tables, and then copy the L^AT_EX code generated to paste in the document.

2.2.3 Network-based Feature Extraction

Protein–protein interaction (PPI) networks, where nodes represent proteins and edges represent physical or functional interactions, offer a framework to study gene relationships at the functional level. By mapping genes onto these networks via their protein products, researchers can analyze the topological and functional proximity of candidate cancer genes to known drivers, helping to identify novel rare drivers missed by mutation frequency alone. <https://www.lagelab.org/resources/>

Table 2.1: Comparison of relevant works and their connection to our quantum embedding method

	Core Idea	Relevance
NetSig (Horn et al., 2018)	Identifies cancer driver genes using aggregated mutation signals from direct network neighbors.	Inspires the use of graph-local statistics to infer node relevance; emphasizes network structure for signal detection.
Moment Propagation (2021)	Propagates higher-order statistical moments of mutation scores through biological networks.	Motivates the use of rich local descriptors (moments) beyond mean aggregation; forms the statistical basis for QMME.
This work (2025)	Encodes multi-order neighborhood moments into quantum states for node classification via fully quantum inference.	Introduces a novel quantum approach to graph embedding and classification, extending the ideas above into a quantum paradigm.

2.3 Quantum Computing

Network-based approaches derive their strength from integrating weak mutation signals across interconnected genes rather than relying on strong evidence from individual genes. This demonstrates the value of graph-structured representations as a paradigm for cancer driver gene discovery. However, as biological networks continue to grow in size and complexity, classical methods face increasing challenges in scalability and representational efficiency. This motivates the exploration of quantum computing as an alternative framework for representing and processing such structured data, including PPI networks. In particular, quantum machine learning (QML), has been proposed as a tool with potential for inference tasks [4, 37–39]. Recent work has begun applying quantum computing to biological network inference problems, including link prediction tasks [40–42], demonstrating potential speedups over classical baselines, further supporting the potential of quantum approaches in this domain. Classical methods often rely on local statistical descriptors, such as neighborhood means or low-order moments, to capture structural information in graphs [2, 43]. However, quantum embedding strategies that incorporate such statistics remain unexplored. Moreover, many existing QML approaches neglect the gate complexity of state preparation, limiting their practicality as end-to-end quantum solutions [44, 45]. Motivated by these gaps, we introduce a quantum protocol that embeds low-order statistical moments of features in the one-hop and two-hop neighborhoods into amplitude-encoded quantum states. Quantum computation is based on two quantum phenomena: quantum interference and quantum entanglement [46].

2.3.1 Qubits and Quantum Operations

In classical computing, information is encoded in bits, which represent binary states — either 0 or 1. Quantum computing, on the other hand, uses quantum bits, or qubits, as its fundamental unit of information. A qubit is also a two-level system, similar to a classical bit, but with a key difference: due to the principles of quantum mechanics, a qubit can exist in a superposition of these two states. This means that a qubit can simultaneously be in a combination of $|0\rangle$ and $|1\rangle$, with corresponding complex probability amplitudes, α and β , such that the state normalization condition $|\alpha|^2 + |\beta|^2 = 1$ holds true.

PM: Enjoyed the Quantum-based representation of states explanation here [\[47\]](#). Great ref for everything [\[48, 49\]](#)

The state of a qubit is represented in Dirac or bra-ket notation as:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad (2.1)$$

where α and β are complex numbers. A powerful way to visualize this quantum state is by re-expressing it in spherical coordinates, leading to the Bloch sphere representation. In this framework, the qubit state is given by:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle, \quad (2.2)$$

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad (2.1)$$

$$|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (2.2)$$

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

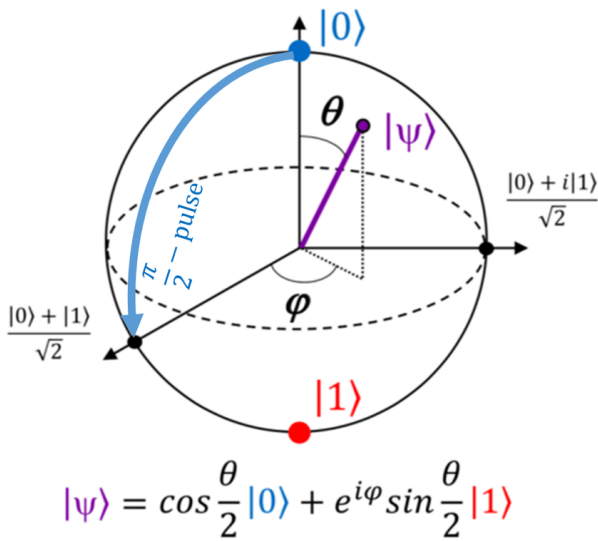
$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = |+\rangle$$

where $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi]$, providing a geometric representation of the qubit state on the Bloch sphere. Include R_X, R_Y, R_Z (most important for me is the R_Y , which allows rotations around the Z axis, and that is the circumference where we are acting)

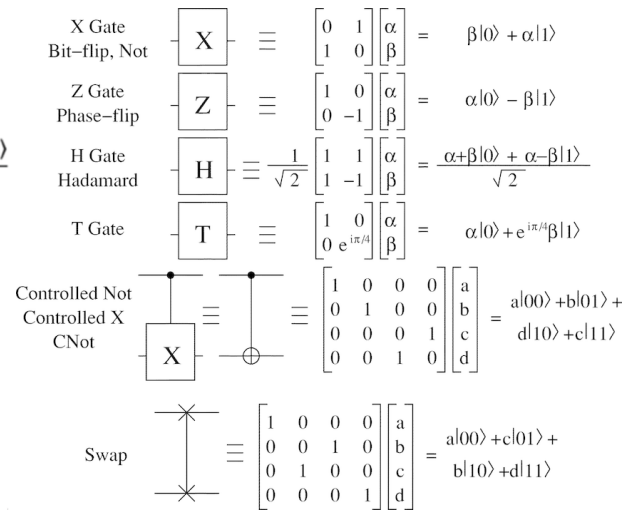
This ability of qubits to exist in superposition allows quantum computers to perform certain computations exponentially faster than classical computers, particularly for problems involving parallelism and complex state manipulations.

PM: Complexity: Now, once we fix a universal set (any universal set) of quantum gates, we'll be interested in those circuits consisting of at most $p(n)$ gates from our set, where p is a polynomial, and n is the number of bits of the problem instance we want to solve. We call these the polynomial-size quantum circuits. <https://www.scottaaronson.com/democritus/lec10.html>. Also, include Figure 7. Classical and quantum data-lookup oracles [60]. from 2024 QML review <https://iopscience.iop.org/article/10.1088/1361-6633/ad7f69/pdf>

Amplitude encoding as rigorously described in [4] (Weighted nearest neighbors (parameter-free binary classification) arxiv.org).



(a) Bloch sphere: Adaptation System Test 4



(b) Quantum gates: Adaptation System Test 5

Figure 2.2: Bloch sphere (images side-by-side)

Both Figures 2.2(a) and 2.2(b)“perfect” tincidunt.

2.3.2 Gate-Based Quantum Computing

Every quantum operator (‘gate’) can be analytically expressed in the form of a matrix [47].

Summarize quantum computing in one sentence, it would be this: information is encoded in quantum states and processed using unitary operations. The challenge of quantum algorithms is to design and build these unitaries to perform interesting and useful tasks with the encoded information. My colleague Nathan Wiebe once told me that some of his early research was motivated by a simple question: Quantum computers can implement products of unitaries — after all, that’s how we build circuits from a universal gate set. But what about sums of unitaries?

Quantum algorithms can be described by unitary transformations and projective measurements acting on the 2^n -dimensional state vector of the register. In this context, unitary transformations are also

called quantum gates [50].

In [51], they enumerate detailed descriptions of the quantum computation model, and outline the basics using the terminology of quantum networks.

Feature Rotation. “Oracles”. Query complexity. Table 2.2 illustrates the use of a Spreadsheet-like table producing calculations by columns and by lines (observe the code).

Table 2.2: A nice Spreadsheet using package “spreadtab”. Notice the calculations.

22	54	76
49	37	86
49	37	86
120	128	248

2.3.3 Block-Encodings

1. **Simulator-Friendly Encodings:** Designed for efficient simulation on quantum computers, often using techniques like qubitization and Hamiltonian simulation.
2. **Block Encodings via Linear Combination of Unitaries (LCU):** Represents a matrix as a weighted sum of unitary operations. Requires ancilla qubits and controlled unitary operations. Commonly used in Hamiltonian simulation and quantum algorithms.
3. **Block Encoding with Matrix Access Oracles:** Useful for sparse and structured matrices. Relies on oracle access to the matrix elements. Can be implemented efficiently in graph-based quantum algorithms.
4. **Sparse Matrix Block Encoding:** Assumes the matrix has a limited number of nonzero entries per row. Uses query-based access to efficiently construct the encoding. Efficient for quantum machine learning and differential equation solvers.
5. **Tensor Product Block Encoding:** Encodes a large matrix by decomposing it into tensor products of smaller matrices. Often applied in quantum chemistry and quantum field theory.

PM: Block-encoding classical data into a quantum circuit has become a central component of modern quantum algorithms. (as in arXiv:2401.04234v1)

The task of constructing a block-encoding is nontrivial. In this paper we pose a different question; how can we efficiently transform application-relevant matrices into Walsh-Hadamard sparse matrices which compress well with FABLE? We begin by noting that generic sparse matrices are ubiquitous in quantum computing algorithms” “While the empirical contraction of quantum resources used by S-FABLE over FABLE is promising, both methods still require a large amount of classical pre-processing to compute the angles used in the rotation gates of the oracle” “For example, S-FABLE and LS-FABLE have difficulty block-encoding graph adjacency matrices with nonzero entries equal to one, and even more disappointingly unstructured nonnegative sparse matrices, requiring nearly all of their rotation gates to block-encode these classes of matrices to moderate accuracy.

Why making a quantum computer is extremely difficult? [47, 52] As inspecting the internal system state inevitably involves some physical interaction with the system itself, an important consequence of the indetermination principle is that any measurement operation on a quantum computer disrupts coherence [47].

Quantum machine learning (QML) approaches can be categorized into two main types: fully quantum computing-based methods and hybrid approaches that combine quantum computing components with classical neural networks. In general, a QML model which works with classical data requires three distinct blocks (Benedetti et al., 2019): data encoding, a variational circuit, and the measurement stage. The data encoding maps classical input data onto qubits, making it accessible to the quantum circuit. The variational circuit incorporates tuneable operations that adjust its parameters to fit the desired function. Finally, the measurement stage transforms the quantum state back into a real vector. Schuld and Petruccione, 2021 propose optimizing the measurement basis to learn functions, while Lloyd et al., 2020 suggests optimizing the variational circuit to maximize the distance between classes of points for classification tasks. Here, Lloyd et al. demonstrate the accuracy of their approach by replacing the fully connected layer of a convolutional neural network. Both approaches enable the realization of parameterized, optimizable functions on quantum circuits.

Given a labeled training set $\{(\mathbf{x}_m, y_m)\}_{m=1}^M$ with $y_m \in \{0, 1\}$ (or $\{-1, +1\}$ under margin-based conventions), and learned or fixed weights $w_m \in \mathbb{R}$, the quantum classifier defines a kernel via a quantum feature map:

$$|\tilde{\mathbf{x}}\rangle = U_\phi(\mathbf{x})|0\rangle,$$

which embeds classical inputs into a quantum Hilbert space via the unitary map U_ϕ .

This choice corresponds to the linear kernel, $\kappa(x, x') = \langle \psi_x | \psi_{x'} \rangle = x^T x'$ (more about quantum kernels in Section 2.3.4.A).

In the Fault-Tolerant Quantum Computation (FTQC) era, we envision a future where we have fully

implemented quantum error correction (great QML 2024 review) []. The framework we explore for efficient block-encoding of sparse and structured matrices is block encoding with matrix access oracles. We assume the existence of the oracles - mention oracles as black-boxes. An oracle is the same (I think) as saying accessing qRAM, or not.

We assume that the feature vector entries are queried using an oracle using controlled rotations [53]. - and introduce just the simple example ever ket ket - ket ket to explain what is happening, like accessing the entries of a matrix or simply vector.

$$O_D |i\rangle |0_l\rangle = |i\rangle |D_{ii}\rangle, \quad i \in [N], \quad (2.3)$$

Remark 2.3.1. We assume each diagonal entry of D can be efficiently computed using a classical Boolean circuit of size $O(\text{polylog}(N))$ with $m = O(\text{polylog}(N))$ ancilla bits. In this case, we can construct a quantum circuit with $O(\text{polylog}(N) + l)$ gates and $O(\text{polylog}(N) + l)$ ancilla qubits to implement this oracle O_D [54] (for simplicity assume the l -bit approximation of D_{ii} is exact. which we MUST check).

Remark 2.3.2. We assume each diagonal entry of D can be efficiently computed using a classical Boolean circuit of size $O(\text{polylog}(N))$ with $m = O(\text{polylog}(N))$ ancilla bits. In this case, we can construct a quantum circuit with $O(\text{polylog}(N) + l)$ gates and $O(\text{polylog}(N) + l)$ ancilla qubits to implement this oracle O_D [54]).

2.3.4 Hybrid Computing

Before that: QGNNs and the difference from my work. Use of Graphs: in my work, Graphs are implicit or used for feature extraction / embedding, in GNNs, Graphs are explicitly encoded in the model architecture (message-passing, graph convolutions, etc.). Quantum Feature Map: in my work, apply a fixed feature map to data (e.g., gene vectors), in GNNs, They define learnable quantum circuits for each node/message-passing operation [55] (QGNNs bible).

A link between the kernel method with feature maps and quantum computation was formally established by proposing to use quantum Hilbert spaces as feature spaces for data [56]. The ability of a quantum computer to efficiently access and manipulate data in the quantum feature space offers potential quantum speedups in machine learning [57] [This paragraph is from [58].] A quantum classifier is similar to a support vector machine that classifies data using learnable linear transformations in an exponentially large feature space.

One popular candidate application for quantum computing is machine learning [59]. Quantum machine learning may be defined as a field that investigates the use of quantum computing as a resource for machine learning [56]. In order for a quantum computer to have an advantage, one would like the

state to be good at describing the solution of interest, while also difficult to prepare and/or sample from classically using currently known methods [60]. Near-term quantum devices require routines that are of shallow depth and robust against errors; the design paradigm of hybrid algorithms which integrate quantum and classical processing has therefore become increasingly important, cited from [61].

The central idea of hybrid algorithms is dividing the problem into two parts, each of which can be performed easily on a classical and a quantum computer [62]. A key point is, in the quantum part of the algorithm, using the quantum state as a feature space (quantum-enhanced feature space) [63]. This is called using quantum feature maps (for supervised learning) [56].

In [63], the algorithms solve a problem of supervised learning: the construction of a classifier. The first method is the quantum variational classifier, uses a variational quantum circuit to classify the data, analogue to conventional SVMs, and corresponds to Section 2.3.4.B. The second method is a quantum kernel estimator, which estimates the kernel function on the quantum computer and optimizes a classical SVM, and corresponds to Section 2.3.4.A.

Note that quantum classifiers cannot provide a quantum advantage over a conventional SVM if the feature vector kernel can be computed efficiently on a classical computer. The authors of [63] highlight that a feature map that is hard to estimate classically is an important part of creating a quantum advantage, and conjecture that the additive error approximation of inner products generated from circuits with two Hadamard layers and diagonal gates is hard classically. That is why Instantaneous Quantum Polynomial Time (IQP) [64] embeddings are typically chosen. An example for a feature map that is believed to be classically intractable is an IQP, as [56] mentions.

In [56], they present mathematical formulations for the first strategy (Section 2.3.4.A) and to the second strategy (Section 2.3.4.B). The implicit approach uses the quantum device to evaluate the kernel function, while in the explicit approach, the model is solely computed by the quantum device. We employ a SWAP test in our approach, following an explicit, closed-form methodology rather than relying on optimization-based techniques, to directly compute overlaps between quantum states.

2.3.4.A Implicit Approach (Kernel Estimator + Classical SVM)

For the general theory of kernel-based quantum binary classifiers, [4] (very useful, including the abstract). Binary classification is an example of pattern recognition. The goal of this task can be described as, given a complex-valued labelled data set $D = (x_1, y_1), \dots, (x_M, y_M) \subset \mathbb{C}^N \times \{0, 1\}$, finding the most likely class label of an unseen data point $x \in \mathbb{C}^N$ [4].

The implicit approach entails two steps: kernel estimation, via a quantum circuit, and a posterior classical classifier, as an SVM. Encode the training data $(y_v, |v\rangle)$ into a quantum feature space, using a quantum kernel function to classify the data [56, 65]. After this feature space mapper, use classical classifiers/techniques (like SVM) to perform the final classification [63]. Apply classical techniques to

extract the final scores. Extract useful information by projecting

$$\sum_{v,c,i} U(v,c,i) |v\rangle |c\rangle |i\rangle \quad (2.4)$$

onto a computational basis. Apply controlled unitary transformations to weight different features, e.g., a weighted sum over the amplitudes corresponding to different (c, i) combinations:

$$\sum_v \left(\sum_{c,i} f(c,i) U(v,c,i) \right) |v\rangle, \quad (2.5)$$

where $f(c, i)$ is a weighting function, like feature importance.

The kernel function measures the closeness of the test data to training data. With a nonlinear kernel, given a quantum test state, the protocol calculates the weighted power sum of the fidelities of quantum data in quantum parallel via a swap-test circuit followed by two single-qubit measurements.

Fidelity measures how close the transformed state is to the target state, and can be used as a transformation performance metric:

$$F(|y\rangle, U|\psi_{\text{final}}\rangle) = |\langle y|U|\psi_{\text{final}}\rangle|^2 \rightarrow 1 - P(\text{error}) \quad (2.6)$$

If fidelity is close to 1, the transformation is accurate. If the fidelity is low, the operator W should be adjusted to improve the transformation. This is typically done using quantum gradient descent, with a parameterized quantum circuit.

Quantum Amplitude Amplification, if the train labels y_v are available, to align $|U\rangle$ with $|y\rangle$.

Classical Post-Processing: Process the measurements using classical methods, e.g., applying a matrix M of size $N \times 8N/256N^3$ to derive the final scores for each node, e.g., post-aggregating the 8 feature values into class predictions.

Hadamard Classifier [66]:

$$|\psi^h\rangle = \frac{1}{\sqrt{2}} \sum_v \frac{1}{\sqrt{N}} \left(|0\rangle |\vec{u}_v\rangle + |1\rangle |\vec{\bar{u}}\rangle \right) |y_v\rangle |v\rangle. \quad (2.7)$$

We don't simply apply the O_L oracle as in ???. Instead, on the new ancilla/control qubit q_c , apply an Hadamard. Then, apply a controlled version of O_L providing that the control qubit q_c is in state $|0\rangle$, such that $O_L : O_L |l\rangle |v\rangle |0\rangle = |r(v, l)\rangle |v\rangle |0\rangle$. Apply a controlled $\tilde{O}_L$ on q_c being 1, which gives the neighbors of v^{unseen} $\tilde{O}_L : \tilde{O}_L |l\rangle |1\rangle = |r(v^{\text{unseen}}, l)\rangle |1\rangle$.

The Hadamard test is a method capable of evaluating such a quantity on a quantum computer using just one additional qubit. This involves applying a control qubit with control over the extra qubit and applying the target unitary operator U to the quantum state. The idea is to apply a controlled U operation,

with the control on that qubit, and target U on the quantum state. Then, one can measure from this single qubit state both the real and imaginary parts of the overlap [45, 67].

The classification is then determined through the expectation value of a two-qubit observable:

$$\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle = \sum_{m=1}^M w_m y_m \Re(\langle \tilde{\mathbf{x}} | \tilde{\mathbf{x}}_m \rangle), \quad (2.8)$$

where $\sigma_z^{(a)}$ and $\sigma_z^{(l)}$ are Pauli-Z operators acting on the ancilla and label qubits, respectively. The inner product $\langle \tilde{\mathbf{x}} | \tilde{\mathbf{x}}_m \rangle$ is evaluated using a Hadamard test, giving access to its real component.

Swap-Test Classifier [58]:

$$|\psi^h\rangle = \sum_v \frac{1}{\sqrt{N}} |0\rangle |\vec{u}_v\rangle^{\otimes n} |\vec{u}\rangle^{\otimes n} |y_v\rangle |v\rangle \text{ with } n \in \mathbb{N} \text{ (polynomial kernel)} \quad (2.9)$$

$$= \sum_v \frac{1}{\sqrt{N}} |0\rangle |\vec{u}_v\rangle |\vec{u}\rangle |y_v\rangle |v\rangle \text{ for } n = 1 \text{ (simplest kernel)}. \quad (2.10)$$

Add five extra ancilla qubits for the controlled rotations to block-encode $\vec{u}$. Use the $2n$ qubits (or more) to similarly apply the embedding scheme to achieve $|\vec{u}\rangle$.

PM: Building a kernel or inner product: measure or interfere only the $|0\rangle$ branch.

The article [68] did not "invent" the SWAP test, but helped making it a standard tool in quantum information theory. The SWAP test classifier is directly based on that, i.e., estimating the fidelity (i.e., the inner product) between two quantum states.

For a realistic hardware application of a controlled-SWAP gate, see [69], where the authors utilizes a hardware efficient circuit quantum acoustodynamic system to implement these gates (copied this from [45]).

Consider a binary classification function

$$f : \mathbb{R}^d \rightarrow \{0, 1\}, \quad (2.11)$$

which maps an input feature vector $\mathbf{x}^{(v)} \in \mathbb{R}^d$ to a predicted label $\hat{y}^{(v)} \in \{0, 1\}$. The prediction rule is structurally similar to classical kernel-based classifiers such as the Support Vector Machine (SVM).

2.3.4.B Explicit Approach (Variational/Parameterized Quantum Circuits)

An interesting class of hybrid quantum-classical algorithms is called variational or parameterized quantum circuits. These can be trained with the help of a classical coprocessor [59]. Possibly the most well-known class of hybrid algorithms is that of variational circuits, which are parameter-dependent quantum circuits that can be optimized by a classical computer with regards to a given objective, cited from [61].

The quantum computer prepares the state, and a classical optimizer tunes the parameters.

In a training-based approach, i.e., quantum machine learning, W or $U(\theta)$ is a trainable quantum operator, a learnable quantum circuit. The set of learnable parameters θ are trained using hybrid quantum-classical optimization, e.g., using quantum gradient descent. The idea is to train the W parameters to solve ???. W will use rotation gates, making this a Parameterized Quantum Circuit (PQC) learning problem. We optimize the weights through trial and error.

In [59] (circuit-centric quantum classifiers), they investigate a class of variational circuits for supervised machine learning, and propose a circuit architecture suitable for predicting class labels of quantumly encoded data via measurements of certain observables. The predicted class is retrieved by measuring a certain qubit in the state $U(\theta)|x\rangle$, with x the input. These quantum classifiers use entanglement as a key resource.

In [62], there is a "new hybrid framework, called quantum circuit learning (QCL), for machine learning with a low-depth quantum circuit. "In QCL, we provide input data to a quantum circuit and iteratively tune the circuit parameters so that the optimized circuit gives the desired output." This QCL framework aims to perform supervised or unsupervised learning tasks.

In [70], something a bit different: the basis of training set labels, after an algorithm they explain, the expansion coefficients of the new state are the desired support vector machine parameters (in the training part). Quantum Support Vector Machines (QSVMs) (might) use quantum kernels for classification. In the test part, they construct a state that is a superposition of the $|\tilde{u}\rangle$ (basically the SVM parameters in the basis) and of the $|\tilde{x}\rangle$ (input data). They then perform a swap test and measure with a success probability P . Then, if P is larger or smaller than $1/2$ determines whether the label is $+1$ or not.

In [61], PennyLane differentiates between classical and quantum nodes for the calculation of gradients. A classical node is a numerical computation, while a quantum node executes a variational circuit U on a quantum device and returns an estimate of the expectation value of an observable $\hat{B}$, estimated by averaging $\hat{R}$ measurements.

PM: There are more ways to calculate gradients (e.g., finite differences). See Note's Chapter 9 for more.

Variational Principle and Quantum Variational Optimization

The Variational Theorem/Principle states that, for any trial wavefunction $|\psi_{\text{trial}}\rangle$, the expectation value of the Hamiltonian H is always greater than or equal to the ground state energy E_0 :

$$\langle\psi_{\text{trial}}|H|\psi_{\text{trial}}\rangle \geq E_0. \quad (2.12)$$

This allows us to approximate the ground state energy E_0 by optimizing a parameterized trial state $|\psi_{\text{trial}}(\vec{q})\rangle$. We adjust parameters $\vec{q}$ to minimize the expectation value:

$$E_{\text{approx}} = \min_{\vec{q}} \langle \psi_{\text{trial}}(\vec{q}) | H | \psi_{\text{trial}}(\vec{q}) \rangle. \quad (2.13)$$

If we find the best choice of $\vec{q}$, the energy E_{approx} is as close as possible to E_0 . This is the foundation of Variational Quantum Eigensolvers (VQE) [60]:

1. Prepare a parameterized quantum state $|\psi(\vec{q})\rangle$.
2. Compute the expectation value $\langle H \rangle$.
3. Use classical optimization to adjust $\vec{q}$ and minimize $\langle H \rangle$.

Assuming N features, we first consider simple normalizing state preparation:

$$(x_1, \dots, x_N)^T \longrightarrow \bar{x} = \frac{1}{\chi} (x_1, \dots, x_N, c_1, \dots, c_P), \quad (2.14)$$

where $c_1, \dots, c_P$ are some padding values that are constant across the data domain, and

$$\chi = \sqrt{\sum_j x_j^2 + \sum_k c_k^2}. \quad (2.15)$$

The padding dimension P is selected such that the total dimensionality becomes a power of 2:

$$N + P = 2^n. \quad (2.16)$$

Amplitude encoding: a normalized input vector $x = (x_0, \dots, x_{N-1})^T \in \mathbb{R}^N$ is encoded such that

$$U_\phi(x) : x \mapsto |\psi_x\rangle = \sum_{i=0}^{N-1} x_i |i\rangle, \quad (2.17)$$

3

Quantum Embedding Protocol

Contents

3.1	Introduction	27
3.2	Input Model	30
3.3	Quantum Multi-order Moment Embedding (QMME) Algorithm	32
3.4	Complexity Analysis	37

3.1 Introduction

3.1.1 Notation

Protein-protein interaction (PPI) networks model interactions between genes as nodes connected by edges as in fig. 1.1. Each gene's association with cancer can be quantified using the MutSig P -value [35, 36], which reflects the statistical significance of mutations considering mutational burden, clustering, and functional impact [2]. Formally, we represent a PPI network as an unweighted, undirected graph $\mathcal{G} = (V, E, \mathbf{x})$ with $|V| = N$ nodes, where $V = [N]$ denotes the set of node indices and $E \subseteq V \times V$ is the edge set. The connectivity of $\mathcal{G}$ is described by its adjacency matrix $A \in \{0, 1\}^{N \times N}$, where $A_{vu} = 1$ indicates the presence of an edge between nodes v and u , and $A_{vu} = 0$ its absence. Define the immediate neighborhood of v as

$$\mathcal{N}_v^1 = \{u \in V \mid A_{vu} = 1\}. \quad (3.1)$$

We assume $\mathcal{N}_v^1$ is ordered by increasing node index. This allows us to define the function $r(v, \ell)$ as

$$r(v, \ell) := \mathcal{N}_v^1(\ell), \quad \ell \in \{0, \dots, |\mathcal{N}_v^1| - 1\}, \quad (3.2)$$

which returns the index of the ℓ -th neighbor of v in $\mathcal{N}_v^1$. Equivalently, $r(v, \ell)$ is the column index of the ℓ -th nonzero entry in row v of A . If $\ell \geq |\mathcal{N}_v^1|$, i.e., the function returns a flag value indicating no such neighbor exists. The two-hop neighborhood $\mathcal{N}_v^2 \subseteq V$ is defined as

$$\mathcal{N}_v^2 = \{w \in V \mid \exists u \in V, A_{vu} = 1 \wedge A_{uw} = 1\}. \quad (3.3)$$

Each node $v \in V$ has a scalar feature value $x_v \in \mathbb{R}$, representing its MutSig P -value quantifying the gene's mutation significance. The collection of all node features is denoted by the vector

$$\mathbf{x} = (x_1, x_2, \dots, x_N) \in \mathbb{R}^N. \quad (3.4)$$

3.1.2 Node Embeddings

The low-order origin (or raw) moments of a scalar random variable $X \sim \mathcal{P}$ are defined as $\mathbb{E}[X^i]$, where i denotes the order of the moment [71]. These moments are uncentered (i.e., calculated about the origin) and capture fundamental properties of the distribution. The first four are the mean $\mathbb{E}[X]$, the uncentered quadratic $\mathbb{E}[X^2]$, cubic $\mathbb{E}[X^3]$, and quartic $\mathbb{E}[X^4]$ moments. Origin moments are closely connected to the

moment-generating function (MGF) of a random variable X , defined as

$$M_X(t) = \mathbb{E}[e^{tX}], \quad (3.5)$$

where $t \in \mathbb{R}$ is a real parameter. The Taylor expansion of $M_X(t)$ around $t = 0$ generates the origin moments of X , providing a compact summary of the distribution:

$$\mathbb{E}[X^i] = \left. \frac{d^i M_X(t)}{dt^i} \right|_{t=0} = \int_{-\infty}^{\infty} x^i \cdot f(x; \theta_1, \dots, \theta_i) dx, \quad (3.6)$$

where $f(x; \theta_1, \dots, \theta_i)$ is the probability density function of X [43]. While central and standardized moments are commonly used in statistical analysis, origin moments are more naturally compatible with our encoding scheme. Although they do not isolate distributional shape in the same way as central moments, they are sufficient for the tasks considered here. Low-order origin moments serve as the building blocks for the more familiar summary statistics of variance, skewness, and kurtosis, which are centered or standardized variants derived from them. Formally, we define the *origin moment embedding* as the mapping

$$\bar{\varphi}(X) = [\mathbb{E}[X], \mathbb{E}[X^2], \mathbb{E}[X^3], \mathbb{E}[X^4]], \quad (3.7)$$

which collects the first four origin moments of X . Since the true distribution $\mathcal{P}$ is generally unknown, these moments are typically estimated empirically. For a finite sample $\mathbf{s} = [s_1, \dots, s_k]$ of X , with $s_j \in \mathbb{R}$, the corresponding *sample raw moment embedding* is given by

$$\varphi(\mathbf{s}) = \left[\sum_{j=1}^k \frac{s_j}{k}, \sum_{j=1}^k \frac{s_j^2}{k}, \sum_{j=1}^k \frac{s_j^3}{k}, \sum_{j=1}^k \frac{s_j^4}{k} \right]. \quad (3.8)$$

which returns the first four sample origin moments from the sample, thus, describing its distributional shape. The hop order $c \in \{1, 2\}$ specifies the neighborhood distance from node v . Given $\mathbf{x}$ defined in eq. (3.4),

$$\mathcal{X}_v^c = \{x_u \in \mathbf{x} \mid u \in \mathcal{N}_v^c\}, \quad (3.9)$$

denotes the set of feature values of nodes at hop-distance c from v , which are samples from a local distribution $\mathcal{P}_v^c$. The function $\varphi(\cdot)$ is applied to each $\mathcal{X}_v^c$, such that

$$\mathbf{m}_v^c := \varphi(\mathcal{X}_v^c) \in \mathbb{R}^4. \quad (3.10)$$

For vertex $v \in V$, the node embedding is then the concatenation of $\mathbf{m}_v^c$ from immediate and two-hop neighborhoods:

$$\mathbf{m}_v = [\mathbf{m}_v^1, \mathbf{m}_v^2] \in \mathbb{R}^8, \quad (3.11)$$

defining the *node embedding function* $\mathbf{m}_v : V \rightarrow \mathbb{R}^8$. These node embeddings can equivalently be expressed in matrix form. For instance, the mean of the feature values of the immediate neighbors of node v , denoted $\mathbb{E}[\mathcal{X}_v^1]$, can be written as

$$\mathbb{E}[\mathcal{X}_v^1] = \mathbf{e}_v K^{-1} A \mathbf{x}, \quad (3.12)$$

where A is the adjacency matrix, $K = \text{diag}(|\mathcal{N}_1^1|, \dots, |\mathcal{N}_1^N|)$ is the degree matrix, and $\mathbf{e}_v \in \mathbb{R}^N$ is the standard basis vector with a one in position v and zeros elsewhere. The raw moments for $i = 2, 3, 4$ are expressed analogously using elementwise powers of $\mathbf{x}$:

$$\mathbb{E}[(\mathcal{X}_v^c)^i] = \frac{1}{|\mathcal{N}_v^c|} A^c \mathbf{x}^{\odot i}. \quad (3.13)$$

Here, $\mathbf{x}^{\odot i}$ denotes the vector obtained by raising each entry of $\mathbf{x}$ to the power of i , i.e., elementwise multiplication repeated i times. This matrix representation clarifies how the node embedding function relates directly to linear algebra operations on the adjacency matrix and feature vector. Since A^c sums over all walks of length c , nodes may be revisited along edges (backtracking), including the starting node itself, and multiple traversals of edges contribute to the totals.

3.1.3 Problem Statement

We aim to develop a quantum algorithm for node-level binary classification on graphs, with explicitly interpretable quantum embeddings. In the quantum computing framework, classical input data $x \in X$ is encoded into quantum states $|\phi(x)\rangle$ in a Hilbert space $\mathcal{H}$. This procedure defines a *quantum feature map* $\phi : X \rightarrow \mathcal{H}$ [56, 59, 63], which associates each classical data point with a unique quantum state. From a circuit perspective, this mapping is implemented via a unitary operator $U_\phi(x)$ acting on the ground state $|0 \cdots 0\rangle$:

$$U_\phi(x) |0 \cdots 0\rangle = |\phi(x)\rangle. \quad (3.14)$$

We refer to $U_\phi(x)$ as the feature-embedding circuit, as it explicitly prepares the quantum state corresponding to the feature map. In much of the quantum machine learning literature, quantum feature maps are treated as black boxes: $U_\phi(x)$ is designed to generate a high-dimensional state whose inner products define a kernel, but the state itself is not directly interpretable [72]. In contrast, our approach constructs quantum states that explicitly encode statistical moments of the graph neighborhood features. Each moment is computed using the adjacency matrix A and feature vector $\mathbf{x}$, and then embedded directly into the quantum amplitudes. This provides an interpretable feature map, rather than relying only on implicit similarity measures.

Concretely, in our setting the parameters of $U_\phi(x)$ depend solely on the network features and con-

nectivity, which are fixed for a given graph input. We therefore denote this unitary as $U_{\mathcal{G}}$. The operator $U_{\mathcal{G}}$ thus serves as a concrete implementation of the abstract quantum feature map $\phi(x)$, specialized to graphs.

We consider a supervised binary classification task on a graph $\mathcal{G} = (V, E, x)$, where a subset of nodes $V_l \subseteq V$ is labeled. For these nodes, $y_v = 1$ for $v \in V_1 \subset V_l$ and $y_v = 0$ for $v \in V_0 \subset V_l$, corresponding to the positive and negative classes, respectively. This is a node-level classification task, i.e., the aim is to predict labels for individual nodes. Our goal is to predict the label $\hat{y}_v$ for any node $v \in V$, using a method that combines a *quantum node embedding* unitary operator $U_{\mathcal{G}}$, derived from the node features x and the connectivity E of $\mathcal{G}$, with a quantum binary classifier $\mathcal{C}$. This combination is used to predict whether a node v belongs to the positive or negative class. That is,

$$\mathcal{C} \left(U_{\mathcal{G}} |v\rangle |0\rangle^{\otimes m} \right) = \hat{y}_v, \quad (3.15)$$

where $\hat{y}_v \in \{0, 1\}$ denotes the predicted class label for node v , and m is the number of data qubits.

The objective is to explicitly define $U_{\mathcal{G}}$ as the state preparation step and design the full quantum circuit for the supervised classification of cancer driver genes. The input is a PPI network where the node features x are per-gene MutSig P -values. For the labeled data, a set of positively labeled genes V_1 is obtained from CGC data, so the positive class corresponds to cancer driver genes.

3.2 Input Model

We base our algorithm on having query access to the graph $\mathcal{G} = (V, E, x)$ through the adjacency list model for bounded-degree graphs [73–75]. In this input model, the graph has N vertices and bounded degree $s = 2^a$, i.e., at most s nonzero entries in each row of the adjacency matrix [53, 76]. To accommodate an attributed network, we extend this model by adding a feature query. Thus, the three available query operations are the following:

1. **neighbor query:** given $v \in V$ and $\ell \in \mathbb{Z}$, returns the ℓ -th neighbour of v if $\ell \leq k_v$, and $*$ otherwise,
2. **feature query:** given $v \in V$, returns the value x_v ,
3. **degree query:** given $v \in V$, returns the degree k_v .

We fix an ordering of the neighbors of each node v , so that $\mathcal{N}_v^1(\ell)$ denotes the ℓ -th neighbor. A quantum equivalent of this input model can be described by defining three corresponding unitary operators O_L , $\tilde{O}_X$, and $\tilde{O}_K$, acting on appropriate Hilbert spaces. Here, O_L provides access to a node's immediate neighborhood, $\tilde{O}_X$ encodes node features, and $\tilde{O}_K$ allows access to the number of c -hop neighbors. One can also view O_L as an oracle that provides access to the non-zero entries of the adjacency matrix,

i.e., the adjacency list of the graph. The so-called *oracles* act as:

$$O_L : |v\rangle |\ell\rangle \mapsto |v\rangle |r(v, \ell)\rangle, \quad (3.16)$$

$$\tilde{O}_X : |v\rangle |0^{d'}\rangle \mapsto |v\rangle |\tilde{x}_v\rangle, \quad (3.17)$$

$$\tilde{O}_K : |v, c\rangle |0^{d'}\rangle \mapsto |v, c\rangle |\tilde{k}_v^c\rangle, \quad (3.18)$$

where v is the node index, ℓ the neighbor index, $\tilde{x}_v$ the d' -bit fixed point representation of the node feature value $x_v \in \mathbf{x}$, and $\tilde{k}_v^c$ the binary representation of the node degree $k_v^c = |\mathcal{N}_v^c|$, i.e., the number of c -hop neighbors. Increasing d' allows higher precision at the cost of more complex implementation [3]. Here, as defined in eq. (3.2), $r(v, \ell)$ denotes the ℓ -th non-zero entry in the v -th row of the adjacency matrix A , i.e., the ℓ -th neighbor of node v , or a flag if no such entry exists [76]. Although $\ell \in [s]$, this index is expanded into an $\lceil \log N \rceil$ -qubit state [76]. For instance, one may let ℓ take the last $\log s$ qubits of the register, following the binary representation of integers. This ensures that all possible neighbor indices are properly encoded when applying the oracle.

In practice, we assume a binary representation of these quantities. Conversion to this binary form should happen outside the quantum gates; inside the circuit, we only manipulate the d' -bit encoded version. If exact indexing is required, one can use $\lceil \log_2(\text{domain size}) \rceil$ -bit integers with appropriate scaling. Alternatively, for fractional values, one can use at least $\lceil \log_2(\text{domain size}) \rceil$ fractional bits, accepting small rounding errors. Increasing d' allows higher precision at the cost of a more complex implementation [3].

Although oracle implementation on quantum hardware remains an active topic of research [77], our work assumes coherent access, allowing queries to be made in superposition, which is a standard assumption in the theoretical framework of quantum algorithms. A necessary condition for this model is reversibility: one must be able to invert the operation, i.e., $O_L^\dagger |r(v, \ell)\rangle |v\rangle = |\ell\rangle |v\rangle$. We also assume access to the controlled versions of the oracles O_L , $\tilde{O}_X$, and $\tilde{O}_K$ and their inverses. Since these oracles are unitary and represented by real-valued matrices, their inverses reduce to their transposes. Neighbor and feature queries, whether classical or quantum, are counted as $\mathcal{O}(1)$ in query complexity [42]. The degree query can be emulated by $\mathcal{O}(\log s)$ neighbor queries via binary search [40, 74]. We introduce $\tilde{O}_K$ as a generalization of the degree query, enabling access to the size of a c -hop neighborhood, with $c \in \{1, 2\}$. Thus, $\tilde{O}_K$ can be constructed from O_L with $\mathcal{O}(\log^2 s)$ queries. A necessary condition for this query model is that O_c is reversible, i.e., we can have $O_L^\dagger |r(v, l)\rangle |v\rangle = |l\rangle |v\rangle$ [76].

3.3 Quantum Multi-order Moment Embedding (QMME) Algorithm

3.3.1 Rotation Operators

The goal of the embedding circuit is to encode local graph statistics into the amplitudes of quantum states. This is done using controlled single-qubit rotations [78], where the rotation angles are determined by values encoded in the computational basis and retrieved from the oracles. To this end, we define two rotation operators, F_X and $F_{K^{-1}}$. First, we start by defining the general controlled rotation operator $W(a)$ as

$$W(a) = \begin{pmatrix} a & -\sqrt{1-a^2} \\ \sqrt{1-a^2} & a \end{pmatrix}. \quad (3.19)$$

The parameter a must lie between -1 and 1 to guarantee the matrix is unitary. This encodes scalar a into 2×2 unitary matrix and corresponds to a y -rotation by the angle $\theta = 2 \arccos a$, i.e.,

$$W(a) = R_y(\theta), \quad \text{with } \theta = 2 \arccos a. \quad (3.20)$$

Given a node feature $x_v \in \mathbb{R}$ with $|x_v| \leq 1$, and its binary encoding $\tilde{x}_v$, the *feature rotation operator* F_X [76] is defined as

$$F_X : |0\rangle |\tilde{x}_v\rangle \mapsto \left(x_v |0\rangle + \sqrt{1-|x_v|^2} |1\rangle \right) |\tilde{x}_v\rangle, \quad (3.21)$$

which corresponds to a single-qubit rotation $R_y(\theta)$ with angle $\theta = 2 \arcsin(x_v)$. This encodes the scalar value x_v into the amplitude of the $|0\rangle$ component. F_X corresponds to a controlled version of $W(x_v)$, such that

$$F_X = \sum_{\tilde{x}_v} W(x_v) \otimes |\tilde{x}_v\rangle \langle \tilde{x}_v|. \quad (3.22)$$

Similarly, we define the *degree-inverse rotation operator* $F_{K^{-1}}$ [54] as

$$F_{K^{-1}} : |0\rangle |\tilde{k}_v^c\rangle \mapsto \left(\frac{1}{\tilde{k}_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{\tilde{k}_v^c} \right|^2} |1\rangle \right) |\tilde{k}_v^c\rangle, \quad (3.23)$$

which can also be expressed as a controlled version of $W\left(\frac{1}{\tilde{k}_v^c}\right)$ constructed as follows

$$F_{K^{-1}} = \sum_{\tilde{k}_v^c} W\left(\frac{1}{\tilde{k}_v^c}\right) \otimes |\tilde{k}_v^c\rangle \langle \tilde{k}_v^c|. \quad (3.24)$$

These operators are implemented using multi-controlled R_y rotation gates [76], where the control register holds the binary input $\tilde{x}_v$ or $\tilde{k}_v^c$. The rotation angles are $\theta(\tilde{x}_v) = 2 \arcsin(x_v)$ and $\theta(\tilde{k}_v^c) = 2 \arcsin(1/\tilde{k}_v^c)$, respectively.

3.3.2 Feature and Degree-Inverse Oracles

To amplitude-encode the node embedding m_v into quantum states, we introduce two derived oracles O_X and O_K^{-1} , which use the basis-encoding input oracles $\tilde{O}_X$ and $\tilde{O}_{K^{-1}}$, along with rotation operators F_X and F_K^{-1} . The *feature oracle* O_X encodes powers of the feature value x_v conditioned on the control register $|i\rangle$. Specifically, the amplitude of the component where the ancilla register is in state $|0\rangle^{\otimes 4}$ is mapped to x_v^i , which relates to the i -th order term in the raw moment embedding function $\varphi(\cdot)$ from eq. (3.8). The oracle O_X acts on a control register, where the integer i in the control register $|i\rangle$ dictates how many of the ancilla qubits are rotated by F_X , as follows:

$$\begin{aligned} |v, i\rangle |0\rangle^{\otimes(4+d')} &\xrightarrow{\tilde{O}_X} |v, i\rangle |0\rangle |\tilde{x}_v\rangle \\ &\xrightarrow{C-F_X} |v, i\rangle \left(x_v |0\rangle + \sqrt{1-|x_v|^2} |1\rangle \right)^{\otimes i} |0\rangle^{\otimes(4-i)} |\tilde{x}_v\rangle \\ &\xrightarrow{\tilde{O}_X^\dagger} |v, i\rangle \left(x_v |0\rangle + \sqrt{1-|x_v|^2} |1\rangle \right)^{\otimes i} |0\rangle^{\otimes(4-i+d')}. \end{aligned} \quad (3.25)$$

Omitting the intermediate register $|\tilde{x}_v\rangle$, fig. 3.1 summarizes the circuit of oracle O_X , which can be expressed as

$$O_X : |v, i\rangle |0\rangle^{\otimes 4} \mapsto |v, i\rangle \left(x_v |0\rangle + \sqrt{1-|x_v|^2} |1\rangle \right)^{\otimes i} |0\rangle^{\otimes(4-i)}. \quad (3.26)$$

The degree-inverse oracle $O_{K^{-1}}$ amplitude-encodes the inverse degree scaling factors $1/k_v^c$, required

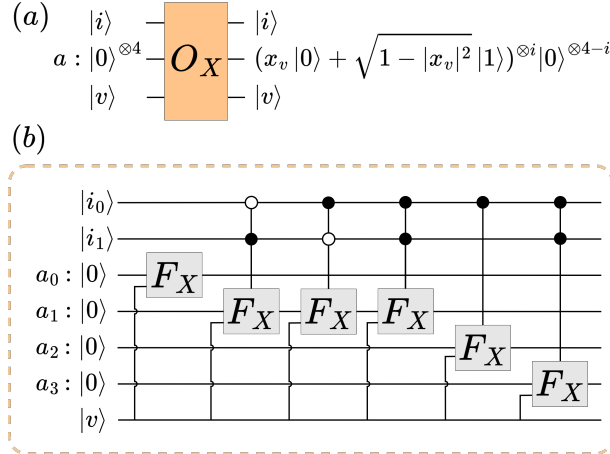


Figure 3.1: Circuit of the feature oracle O_X , built from controlled rotations F_X . (a) Gate representation of O_X . (b) Detailed view: a two-qubit control register $|i\rangle$ determines how many of the four ancilla qubits undergo F_X rotations (00 $\rightarrow$ first qubit, 01 $\rightarrow$ first two, 10 $\rightarrow$ first three, 11 $\rightarrow$ all four). A d' -qubit register is used internally but omitted for clarity.

to emulate the average present in the moment embedding function $\varphi(\cdot)$, included in the node embedding m_v . It performs a controlled rotation that encodes the value $1/k_v^c$ as the amplitude of the $|0\rangle$ state of an

ancilla qubit, as follows:

$$\begin{aligned}
|v, c\rangle |0\rangle^{\otimes(1+d')} &\xrightarrow{\tilde{O}_K} |v, c\rangle |0\rangle |\tilde{k}_v\rangle \\
&\xrightarrow{F_{K-1}} |v, c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v^c} \right|^2} |1\rangle \right) |\tilde{k}_v\rangle \\
&\xrightarrow{\tilde{O}_K^T} |v, c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v^c} \right|^2} |1\rangle \right) |0\rangle^{\otimes d'}.
\end{aligned} \tag{3.27}$$

As shown in fig. 3.2, the intermediate register $|\tilde{k}_v\rangle$ is omitted, and the oracle is represented as:

$$O_{K-1} : |v, c\rangle |0\rangle \mapsto |v, c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v^c} \right|^2} |1\rangle \right). \tag{3.28}$$

Together with the neighbor oracle O_L and Hadamard gates, the oracles O_X and O_{K-1} form the core building blocks of the QMME circuit, as depicted in Figure 3.2. We omit the intermediate registers $|\tilde{x}_v\rangle$ and $|\tilde{k}_v\rangle$, since after reversing the quantum gates (uncomputing), their values will return to zero at the end of the computation.

3.3.3 Embedding Circuit Design

We define the QMME unitary operator $U_{\mathcal{G}}$ as

$$U_{\mathcal{G}} : |v\rangle |0\rangle^{\otimes m} \mapsto |v\rangle |\psi_v\rangle, \tag{3.29}$$

and refer to this mapping as the *quantum node embedding*. The second register, consisting of m qubits, is called the *data register*. The total number of qubits is $m = 2 \log N + 8$ (excluding work registers), and generally scales as $\mathcal{O}(\log N)$. A schematic of the operator $U_{\mathcal{G}}$ circuit is shown in Figure 3.2, and step-by-step derivation of the quantum node embedding in eq. (3.29) is presented in the Appendix. The resulting state $|\psi_v\rangle$ consists of two components:

$$|\psi_v\rangle = |\psi_m\rangle + |\psi_{\perp}\rangle, \tag{3.30}$$

where $|\psi_m\rangle$ is the *neighborhood-moment component*, capturing the quantum analogue of the classical embedding $\mathbf{m}_v \in \mathbb{R}^8$, and $|\psi_{\perp}\rangle$ is an orthogonal component.

According to the circuit in Figure 3.2, $|\psi_m\rangle$ is defined as

$$|\psi_m\rangle = |\mathbf{c} \odot \mathbf{m}_v\rangle |0\rangle^{\otimes(m-3)} \quad (3.31)$$

$$= \sum_{c \in [2]} \frac{1}{2^{3/2} s^c} \sum_{i \in [4]} \varphi_i(\mathcal{X}_v^c) |c, i\rangle |0\rangle^{\otimes(m-3)} \quad (3.32)$$

$$= \sum_{j \in [8]} c_j \cdot m_{v,j} |j\rangle |0\rangle^{\otimes(m-3)}, \quad (3.33)$$

where $\odot$ denotes the element-wise (Hadamard) product, $\varphi_i(\mathcal{X}_v^c)$ are the i th-order moment of the neighborhood $\mathcal{X}_v^c$, and the constant vector $\mathbf{c} \in \mathbb{R}^8$ is

$$\mathbf{c} = \frac{1}{2^{3/2}} \left[\frac{1}{s}, \frac{1}{s}, \frac{1}{s}, \frac{1}{s}, \frac{1}{s^2}, \frac{1}{s^2}, \frac{1}{s^2}, \frac{1}{s^2} \right]. \quad (3.34)$$

The orthogonal component $|\psi_\perp\rangle$ of the resulting state is

$$|\psi_\perp\rangle = \sum_{a \neq 0} |\psi_{\perp,a}\rangle |a\rangle, \quad (3.35)$$

where $|a\rangle$ denotes computational basis states in the $(m-3)$ data qubits, and $|\psi_{\perp,a}\rangle \in \mathbb{R}^{2^{(m-3)}}$.

As an example, take the mean of the mutation scores of the immediate neighbors of gene v , i.e., $m_{v,1} = \varphi_1(\mathcal{X}_v^1)$. As in eq. (3.32), the amplitude of the state $|0_c, 0_i, 0^{(m-3)}\rangle$ will be $\frac{\varphi_1(\mathcal{X}_v^1)}{2^{3/2}s}$. As shown in fig. 3.2, this arises by:

1. preparing an equal superposition over register ℓ_1 ;
2. O_L retrieves the immediate neighbors' indices;
3. operator O_X encodes the neighbors' feature values into the $|0\rangle$ component of the first ancilla qubit;
4. uncomputing the basis register ℓ_1 via O_L^T leaves the $|0\rangle^{\otimes n}$ component amplitude proportional to the sum of neighbors' features, while others form interference patterns with alternating signs;
5. finally, the ancilla rotation O_{K-1} normalizes by the degree k_v^c , resulting in the mean of $\mathcal{X}_v^c$ encoded in the amplitude of the rotated ancillas at zero.

This procedure describes the action of the unitary $U_{\mathcal{G}}$ for $c, i = 0$. It generalizes to $c \in [2], i \in [4]$, covering immediate and second neighbors, and more moments, yielding

$$\langle v, c, i | \langle 0 |^{\otimes(m-3)} U_{\mathcal{G}} | v \rangle | 0 \rangle^{\otimes m} = \frac{1}{2^{3/2} s^c} \sum_{x_u \in \mathcal{X}_v^c} \frac{x_u^i}{k_v^c} = \frac{\varphi_i(\mathcal{X}_v^c)}{2^{3/2} s^c}. \quad (3.36)$$

Without measurements, the final state resulting from the Quantum Multi-order Moments Embedding

Protocol (QMMEP) as full form

$$U_G |v, 0^m\rangle \quad \text{where } m = 2 \log N + 8, \text{ omitting work qubits,} \quad (3.37)$$

$$= |v\rangle \left(\sum_{i,c} \frac{1}{2^{3/2} s^c} \sum_{x_u \in \mathcal{X}_v^c} \frac{x_u^i}{k_v^c} |c, i, 0^{(2n+5)}\rangle + |\perp\rangle \right) \quad (3.38)$$

$$= |v\rangle \left(\sum_{i,c} \frac{1}{2^{3/2} s^c} \varphi(\mathcal{X}_v^c) |c, i, 0^{(2n+5)}\rangle + |\perp\rangle \right) \quad (3.39)$$

$$= |v\rangle \left(\sum_{j \in [8]} c_j m_{v,j} |j, 0^{(2n+5)}\rangle + |\perp\rangle \right) \quad \text{with } c \text{ defined as in eq. (3.34).} \quad (3.40)$$

$$= |v\rangle \left(|\vec{u}_v\rangle |0^{(2n+5)}\rangle + \sum_{a \neq 0^{(2n+5)}} |\vec{\phi}_a\rangle |a\rangle \right), \quad (3.41)$$

where $|\vec{u}_v\rangle = \sum_{j=1}^8 u_{v,j} |j\rangle = \sum_{j=1}^8 c_j m_{v,j} |j\rangle$. Here, the orthogonal component corresponds to "leakage" terms caused by using rotations, based on non-unit amplitudes. In Equation (3.41), observe that the conditional distribution over c (hop index) and i (power index) registers is uniform. That is, each hop $c \in \{1, 2\}$ and each moment $i \in \{1, 2, 3, 4\}$ appears with equal probability. Therefore, the variation in the amplitudes of the full state arises entirely from the node index v . For fixed values of c and i , the amplitude of the state depends solely on the node v , through the features $m_{v,j}$. The encoded data is thus node-specific, while c and i serve as structured axes across which the same statistical moment is applied uniformly to all nodes, as we will do in Chapter 4. The circuit uses the registers described in Table 3.1. To illustrate the use of the neighbor registers l_1 and l_2 in Figure 3.2, notice that the control

Table 3.1: Quantum circuit registers for Quantum Multi-order Moments Embedding (QMME).

Register	Description	Qubit Count	Gate Operations
v	Indexes nodes v	$\log N$	O_L, O_L^T
l_1	Indexes immediate neighbors	$\log s$	$I_{n-s} \otimes H^{\otimes 5}, O_L, O_L^T$
l_2	Indexes second-order neighbors	$\log s$	$I_{n-s} \otimes H^{\otimes 5}, O_L, O_L^T$
c	Control qubit: indexes hop $c \in \{1, 2\}$	1	$H, O_{K^{-1}}$
i	Indexes the i -th moment	2	$H^{\otimes 2}, O_X$
a	Ancilla register for controlled rotations	5	$O_X, O_{K^{-1}}$
work qubits	Basis-encode $\tilde{x}_v$ and $\tilde{k}_v^c$	2d'	Rotation operators (section 3.3.1)

qubit c determines which register is active:

- For $c = 0$, l_1 is put into superposition, O_L retrieves immediate neighbors, and O_X encodes their features.
- For $c = 1$, l_2 is put into superposition, O_L retrieves second-hop neighbors, and O_X encodes their features.

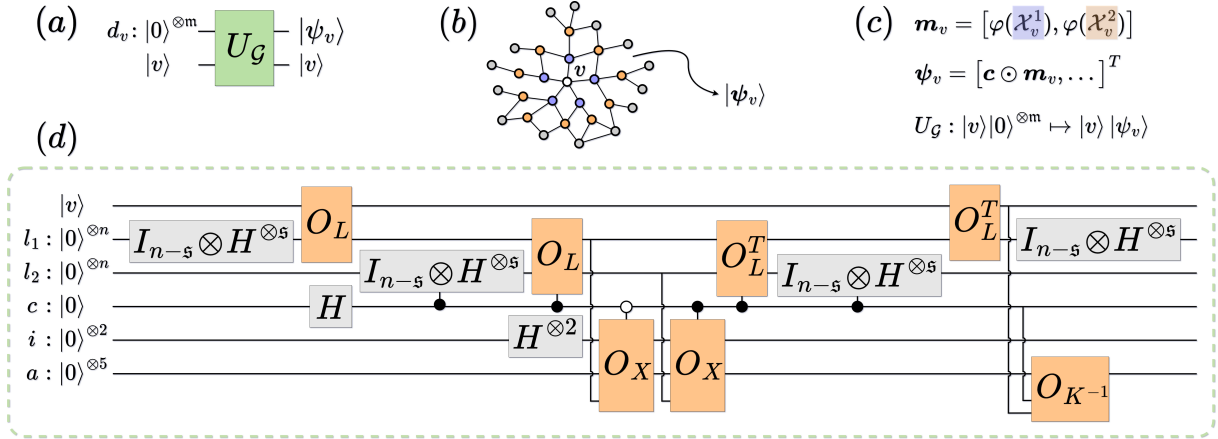


Figure 3.2: Quantum Multi-order Moment Embedding (QMME) protocol. (a) Gate view of the unitary operator U_G , which maps $|v\rangle|0\rangle^{\otimes m} \mapsto |v\rangle|\psi_v\rangle$, amplitude-encoding low-order statistical moments of node v 's neighborhood. Here, $|v\rangle$ encodes the node index $v \in V$, and the m -qubit data register is initialized to zero. (b) Input PPI network $\mathcal{G} = (V, E, \mathbf{x})$ with node features $\mathbf{x}$ representing mutation scores. Immediate neighbors of v are shown in purple, second-hop neighbors in orange. These provide the data for the embedding $|\psi_v\rangle \in \mathbb{R}^{2^m}$. (c) The node embedding function $\mathbf{m}_v$, based on the moments $\varphi(\cdot)$ over v 's neighborhood, is encoded as the quantum state $|\psi_v\rangle$. (d) Circuit-level implementation of the gate U_G . The m -qubit register is composed of subregisters for immediate and second-hop (ℓ_1, ℓ_2) neighbors, a control qubit c , a two-qubit register i indexing the i -th moment, and a five-qubit ancilla register a . The circuit accesses the PPI network in (b) via the quantum oracles in eq. (3.18).

Effectively, when $c = 1$, this procedure implements a controlled version of A^2 , as only second-hop neighbors are accessed. The c register thus ensures that the correct neighborhood order is embedded into the quantum state. We encode $x^{\odot i}$ as $\prod_i U_{\mathbf{x}}$, controlled on the two-qubit register i . This choice reduces the number of oracle calls at the cost of additional rotations and qubits. An alternative would be to encode $x^{\odot i}$ as $U_{\mathbf{x} \odot i}$ with rotation angle $\theta = 2 \arcsin(x_v^i)$, which would require more assumptions on feature rotation operators available (four instead of one) but fewer ancilla qubits (one instead of four needed as described in fig. 3.1).

3.4 Complexity Analysis

A controlled rotation on the ancilla qubit a_1 , conditioned on v , has gate complexity $O(n)$. The hidden overhead comes from the basis encoding of the rotation angle θ_v , which most likely requires $O(\text{polylog } n)$ oracle calls:

$$|x'\rangle|0^n\rangle|0\rangle \longrightarrow |x'\rangle|\arccos(x')\rangle|0\rangle. \quad (3.42)$$

The oracle O_X allows to load feature values, mapping a node index v to its feature value x_v . The cost depends on how x_v is stored. If stored classically and queried efficiently: $O(\text{polylog } N)$. If loaded using qRAM: $O(\log N)$ [39]. If x_v is loaded from a classical lookup table, this can be done in $O(\text{polylog } N)$ elementary gates.

The feature operator F_X is a $C - R_y(\theta)$ gate with $\theta = 2 \arccos x_v$. Compute θ from x_v , then, apply a controlled rotation. Requires computing θ and performing a controlled $R_y(\theta)$, which can be decomposed into [54]:

- Computing $\theta = 2 \arccos x_v$: Quantum arithmetic circuit of $O(\text{polylog } N + d')$ elementary gates. Computed on-the-fly instead of stored. For fast inversion, Proposition 6 [54].
- Controlled $R_y(\theta)$: Can be implemented using **1 CNOT and 2 single-qubit rotations**.

Thus, the **total gate count** is: $O(\text{polylog } N + d')$. Since O_X and F_X are applied sequentially, circuit depth is at most $O(\text{polylog } N + d')$.

In terms of work qubits, O_X requires d' qubits to store x_v , and F_X requires an ancilla qubit for the rotation. Thus, the **total work qubits** required is: $O(d')$.

Oracles that access entries of a diagonal matrix, or entries of a one-dimensional vector with N entries, can be implemented with $O(\text{polylog}(N) + d')$ quantum gates and $O(\text{polylog}(N) + d')$ ancilla qubits Remark 2.3.1. For both O_K and O_x , this implies a total of $O(\text{polylog}(N) + p_x + p_k)$ quantum gates and $O(\text{polylog}(N) + p_x + p_k)$ ancilla qubits to implement these oracles, with $p_x = \text{precision}(\tilde{x})$ and $p_k = \text{precision}(\tilde{k})$ Remark 2.3.1 (for simplicity assume the I-bit approximation of D_{ii} is exact. which we MUST check).

Consider having precomputing basis encoding: feature values (or transformed values $\arccos(x_v)$), i.e., classically computed and stored in a structured quantum state.

Counting the complexity calculations twice, for O_X and O_K ; a number of $n + 4s + 2 + 1$ Hadamard gates adds $O(n + s)$ into the gate count. Since $m \ll n$ and $n = \log N$. About the **number of work qubits**, we need $2 + 1 + n + 2s + 4$, without counting basis-encoding of $\arccos$ values, since we consider those are computed on-the-fly (and then included in gate complexity), and without counting l of the basis encoding of feature and degree values. Thus, the **total work qubits for the QMME circuit** required is:

Our goal is to construct a circuit that has a gate complexity of $\text{polylog}(n)$ at least for certain sparse and/or structured matrices with well defined sparsity patterns [53]. On query calls, it depends on how "complex"/component-based we define them, e.g.: O_X including F_X , or not. But, if it does not, we call each, and its inverse, once, and we query F_X four times, and $F_{K^{-1}}$ once.

Sometimes we are interested in the number of calls to a (usually expensive to compute) function. Such functions are often called black box or oracle functions, since it is assumed that one does not have access to the inner workings of the computation of this function, only to the result of its application.

PM: Write U_A as factored as a product of simpler (Pauli, R_y) unitaries. Section 4 of [53]. “Possible to develop a general scheme to construct a block encoding and the corresponding quantum circuit for well-structured matrices. In particular, when the matrix A is sparse with a structured sparsity pattern and the nonzero matrix elements can also be well characterized, an efficient block encoding circuit for A can be constructed, as we will show in the next section.” “Our goal is to construct a circuit that has a gate complexity of $\text{poly}(n)$, i.e., a polynomial in n , $[\text{polylog } N]$ at least for certain sparse and/or structured matrices with well defined sparsity patterns.” “This assumption is of course somewhat restrictive” “it can generally be more difficult to explicitly construct quantum circuits for O_C in these more general query models.” [meter isto na complexidade]

4

Quantum Classification Protocol

Contents

4.1	Introduction	43
4.2	Quantum Classifier Design	44
4.3	Connection to Classical Models	45
4.4	Complexity Analysis	47

4.1 Introduction

Consider the task of dealing with an unknown quantum state drawn from an ensemble, where each state is associated with a class label. The central question is: how accurately can we predict the class of this unknown quantum state? This problem is broadly known as quantum state discrimination, and in the context of quantum machine learning, it is often referred to as the quantum classification problem [58, 79]. In this work, we explore the possibility of performing the entire classification process using quantum methods. Specifically, we investigate how far one can go under the assumption that (i) the input data is provided in a quantum-encoded format produced by the QMME protocol, and (ii) local node statistics carry sufficient discriminatory information. To this end, we propose a quantum classification strategy including a swap-test, that is training-free and non-iterative, relying only on direct quantum state comparisons rather than parameter optimization.

The state generated by the QMMEP is given by

$$U_G |v, 0^m\rangle \quad (4.1)$$

$$= |v\rangle \left(\sum_{j \in [8]} c_j m_{v,j} |j, 0^{(2n+5)}\rangle + |\perp\rangle \right) \quad \text{with } c \text{ defined as in eq. (3.34).} \quad (4.2)$$

$$= |v\rangle \left(|\vec{u}_v\rangle |0^{(2n+5)}\rangle + \sum_{a \neq 0^{(2n+5)}} |\vec{\phi}_a\rangle |a\rangle \right) \quad (4.3)$$

$$= |v, \psi_v\rangle, \quad (4.4)$$

where $|\vec{u}_v\rangle = \sum_{j=1}^8 u_{v,j} |j\rangle = \sum_{j=1}^8 c_j m_{v,j} |j\rangle$ is the *core* feature vector for node v , j is the feature register for the j -th *core* feature, and $u_{v,j} \in \mathbb{R}$. This is considered input for the Quantum Classification Protocol (QCP). Note the normalization conditions: $\sum_{j=1}^8 |u_{v,j}|^2 = 1$ and that $\sum_{j=1}^8 p(j) = 1$. The QMME state $|v, \psi_v\rangle$ is entangled and cannot be factorized across all registers. Due to the controlled rotations and oracle operations, the ancilla and neighbor qubits are entangled with the feature qubits. The "core" feature vector $|\vec{u}_v\rangle$ is embedded within this structure, forming a superposition but not internally entangled itself.

The quantum node embedding operator U_G maps each node in a feature-labeled network to its statistical moments, amplitude-encoded within a quantum state $|\psi_v\rangle$. Given a protein-protein interaction (PPI) network as input, the QMME protocol encodes local statistical moments of the MutSig P -values. Depicted in Figure 4.1, the quantum classification framework assumes query access to the class labels $y_{v_l} \in \{0, 1\}$ for each labeled node $v_l \in V_l$:

1. **label query:** given $v_l \in V_l$, returns the label y_{v_l} ,

with a quantum equivalent defined by the operator O_Y :

$$O_Y : |v_l\rangle |0\rangle \mapsto |v_l\rangle |y_{v_l}\rangle, \quad (4.5)$$

encoding the class label $y_{v_l} \in \{0, 1\}$ in a single qubit. In practice, this can be realized by applying an $X^{y_{v_l}}$ gate to the state $|0\rangle$ [58].

4.2 Quantum Classifier Design

The proposed quantum classification framework proceeds as follows: after preparing the equal superposition over labeled nodes, the embedding unitary U_G is applied both to this register v_l and to the test node register v_t . The class labels are loaded onto the label qubit via O_Y . Classification follows via the swap-test classifier [58], where the measurement statistics determine the final class prediction of the test node. The full pipeline may be described as a sequence of quantum state evolutions:

$$\begin{aligned} & |0\rangle |0\rangle^{\otimes m} |v_t\rangle |0\rangle |0\rangle^{\otimes m} |0\rangle^{\otimes n} \\ & \xrightarrow{H_{v_l}^{\otimes n}} \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} |0\rangle |0\rangle^{\otimes m} |v_t\rangle |0\rangle |0\rangle^{\otimes m} |v_l\rangle \\ & \xrightarrow{U_G} \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} |0\rangle |\psi_t\rangle |v_t\rangle |0\rangle |\psi_l\rangle |v_l\rangle \\ & \xrightarrow{O_Y} \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} |0\rangle |\psi_t\rangle |v_t\rangle |y_l\rangle |\psi_l\rangle |v_l\rangle \\ & \xrightarrow{H_a \cdot \text{c-swap} \cdot H_a} |\psi_f\rangle = \frac{1}{\sqrt{|V_l|}} \sum_{v_l \in V_l} (|0\rangle |\psi_+\rangle + |1\rangle |\psi_-\rangle) |v_l\rangle, \end{aligned} \quad (4.6)$$

where $|\psi_{\pm}\rangle = |\psi_t\rangle (|v_t\rangle |y_l\rangle) |\psi_l\rangle \pm |\psi_l\rangle (|v_t\rangle |y_l\rangle) |\psi_t\rangle$. The swap-test classifier computes the fidelity, i.e., the squared overlap between two states. This yields $|\langle\psi|\phi\rangle|^2$ for two pure states $|\psi\rangle$ and $|\phi\rangle$, and more generally $\text{Tr}(\rho\sigma)$ for two possibly mixed states ρ and σ . Fidelity serves as a similarity measure (one if the states are identical, zero if they are orthogonal), making it a natural distance measure in the quantum feature space. Classification thus relies directly on distances between training and test data, unifying feature mapping and classification into a single quantum procedure without requiring an intermediate classical support vector machine. Here,

$$\begin{aligned} |v_t, \psi_t\rangle &= |v_t, 0^m\rangle \\ |v_l, \psi_t\rangle &= |v_l, 0^m\rangle, \end{aligned} \quad (4.7)$$

with $v_l \in V_l$ the index of the test node, and $v_l \in V_l$ that of a labeled node.

The measurement and post-processing involves evaluating the expectation value of the two-qubit

observable $\sigma_z^{(a)} \sigma_z^{(l)}$, where the superscripts indicate the ancilla and label qubit. For the final state $|\psi_f\rangle$, $\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle$ is given by

$$\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle = \text{Tr} \left(\sigma_z^{(a)} \sigma_z^{(l)} |\psi_f\rangle \langle \psi_f| \right) \quad (4.8)$$

$$= \frac{1}{|V_l|} \sum_{v_l \in V_l} (-1)^{y_{v_l}} |\langle \psi_t | \psi_{v_l} \rangle|^2. \quad (4.9)$$

In practice, these statistics are obtained by repeating r times the two-qubit projective measurement in the σ_z basis. Using $\text{Tr}(\sigma_z |y_{v_l}\rangle \langle y_{v_l}|) = 1$ for $y_{v_l} = 0$ and -1 for $y_{v_l} = 1$, the expectation value is calculated as

$$\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle = \frac{1}{r} (c_{00} - c_{01} - c_{10} + c_{11}), \quad (4.10)$$

where c_{al} counts measurements with the ancilla in state a and the label qubit in state l . The test data are classified as 0 if $\langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle > 0$ and 1 if negative, i.e.,

$$\begin{aligned} \hat{y}_v &= \frac{1}{2} \left(1 - \text{sgn} \langle \sigma_z^{(a)} \sigma_z^{(l)} \rangle \right) \\ &= \mathcal{C} \left(U_{\mathcal{G}} |v\rangle |0\rangle^{\otimes m} \right) \end{aligned} \quad (4.11)$$

which resembles the earlier formulation in Equation (3.15). Here, $\mathcal{C}$ is the swap-test classifier and $U_{\mathcal{G}}$ corresponds to the Quantum Multi-order Moment Embedding (QMME), encoding local properties in the network $\mathcal{G}$. Conceptually, the swap-test classifier implements a quantum kernel method [80]: each test state is assigned the label whose corresponding training state achieves the largest overlap. A strong analogy is classical k-nearest neighbors (kNN) methods, where the similarity measure in Hilbert space, i.e., the squared overlap $|\langle \psi_t | \psi_{v_l} \rangle|^2$, plays the role of a distance or similarity metric between feature vectors [81].

4.3 Connection to Classical Models

Analogy with Classical Binary Classification: Support Vector Machine (SVM) with a Custom Kernel / But this is a parameter-free model, linear classifier.

In classical machine learning, similarity is typically defined via real-valued distance functions (e.g., Euclidean distance or kernel functions). In the quantum setting, fidelity (or equivalently the state overlap) serves as the natural similarity measure: it takes value one if the states are identical and zero if they are orthogonal. From this perspective, the role of the swap-test is analogous to the kernel or activation function in graph neural networks (GNNs), where feature similarity drives label propagation and classification. Here, however, the similarity is computed natively in the quantum Hilbert space, unifying feature mapping and classification into a single quantum procedure without requiring an intermediate classical

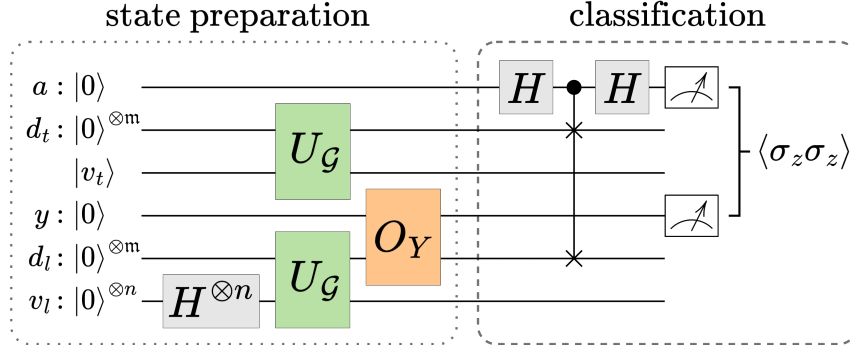


Figure 4.1: Full cancer gene prediction pipeline with the QMME protocol as part of state preparation. The first register corresponds to the ancilla qubit a , the second to the test data qubits d_t , the third to the index qubits of the test node v_t , the fourth stores the label qubit y , the fifth the labeled data qubits d_l , and the sixth the index qubits of the labeled nodes v_l . The operator U_G , together with the Hadamard gate and the oracle O_Y , prepares the quantum state, which is then processed by the classifier $\mathcal{C}$, i.e., the swap-test classifier. Node labels $\hat{y}_v \in \{0, 1\}$ are predicted via a swap-test and two-qubit measurement statistics on the ancilla and label registers. An outcome of label one indicates that the test gene is predicted as a cancer driver.

model.

The decision boundary in this quantum classifier is analogous to that of a classical kernel SVM, which defines a classifier of the form:

$$f(\mathbf{x}) = \sum_{m=1}^M \alpha_m y_m K(\mathbf{x}, \mathbf{x}_m) + b, \quad (4.12)$$

where $K(\cdot, \cdot)$ is a positive semidefinite kernel function, α_m are dual weights learned during training, and b is a bias term. The predicted label is:

$$\hat{y} = \text{sign}(f(\mathbf{x})). \quad (4.13)$$

In this analogy, the quantum kernel

$$K_Q(\mathbf{x}_t, \mathbf{x}_l) := \Re(\langle \tilde{\mathbf{x}}_t | \tilde{\mathbf{x}}_l \rangle)$$

plays the role of the classical kernel K , and the Pauli-Z observable expectation captures the aggregated similarity measure used for classification. The Hadamard classifier thus functions similarly to a classical linear classifier in a high-dimensional feature space, where the kernel is defined by quantum state overlaps rather than explicit analytical functions.

4.4 Complexity Analysis

PM: I absolutely loved the writing on this review, and the clarity of the content. Get inspired by it "Likewise, the 'output problem' is faced when reading out data after being processed on a quantum device. Like the input problem, the output problem often causes a noticeable operational slowdown. (...) With the notable exceptions of least-squares fitting and quantum support vector machines, linear-algebra-based algorithms can also suffer from the output problem because desirable classical quantities such as the solution vector for HHL or the principal components for PCA are exponentially hard to estimate. (...) Ongoing work is needed to optimize such algorithms, provide better cost estimates and ultimately to understand the sort of quantum computer that we would need to provide useful quantum alternatives to classical machine learning." [39]

Whether we have a quantum speedup or not depends on a mathematical proof, if we take the computer science perspective, or on statistical evidence of a scaling advantage, if we take the finite size devices perspective, more low-level or physics view. The latter perspective relies on the existence of a quantum computer - it's a benchmarking problem. Quantum speedups in machine learning (and general) are currently characterized using idealized measures from complexity theory: query complexity and gate complexity [39]. [39] also argues that even very simple circuits are hard to simulate classically.

5

Simulation and Implementation

Contents

5.1	Graph Augmentation	51
5.2	Synthetic Dataset/Toy example	52
5.3	Real Network Data	52
5.4	Complexity Analysis	54

The device can either refer to quantum hardware or a classical simulator [61]. They also say "quantum device (hardware or simulator)".

We represent each node in the network with its $-\log_{10}$ transformed MutSig P-value [2, 36].

Binary string mapping - Big-Endian (normal) - Mathematica: Left-to-right, most significant bit first - Little-Endian (abnormal) - Python: Right-to-left, least significant bit first

5.1 Graph Augmentation

The use of a flag for cases where ℓ exceeds the number of non-zero entries ensures the oracle O_L can handle all possible inputs consistently. For implementation purposes, one must define such a scheme. This scheme can be directly related to the problem of finding the minimal r -regular graph containing an induced subgraph of interest, [82] which, in our case, represents the PPI network.

We introduce M "dummy" nodes, with feature values set to 0, to create an r -regular graph with $N + M$ nodes. The expansion ensures every node, including previously isolated ones, has exactly s different neighbors (from the original PPI net or the dummy nodes). O_L maintains unitarity in this enlarged network. The termination condition for O_K remains valid, returning k_v for the original subgraph.

Upper and lower boundary on m Algorithm 5.1. asterisco $\rightarrow$ graph augmentation (coisa técnica) / outros métodos em vez do graph augmentation (válido).

Algorithm 5.1: Quantum Multi-order Moment Embedding Protocol Strategy

```

begin
   $nextBitrate \leftarrow nextDownloadLevel$ 
   $nextBitrate \leftarrow GetNextBitrate()$ 
   $cpuLoad \leftarrow GetCpuLoad()$ 
   $bitrateDelta \leftarrow getBitrateDelta(currentBitrate, nextBitrate)$ 

  if  $bitrateDelta > maxThreshold$  then
     $SetBitrate(nextBitrate)$ 

  if  $minThreshold < bitrateDelta < maxThreshold$  and  $numAttempts < 2$  then
     $numAttempts \leftarrow numAttempts + 1$ 

  else if  $minThreshold < bitrateDelta < maxThreshold$  and  $numAttempts = 2$  then
     $numAttempts \leftarrow 0$ 
  else
     $SetBitrate(nextBitrate)$ 

  if  $0 < bitrateDelta < minThreshold$  and  $numAttempts < 3$  then
     $numAttempts \leftarrow numAttempts + 1$ 

  else if  $0 < bitrateDelta < minThreshold$  and  $numAttempts = 3$  then
     $SetBitrate(nextBitrate)$ 

```

Multi-source streaming: The app should support multi-source streaming media, i.e., "simultaneous" streaming of media, supported/complemented by QMME/Quantum Classification (QC) [?].

Access Network independency: The solution should provide the expected service over data networks such as QMME, QCP, etc.

Note that, since the “dummy” neighbors of v have feature value 0, the average feature value of the first-neighbor nodes for each of the N nodes of interest is, indeed, encoded in the sub-state amplitude.

5.2 Synthetic Dataset/Toy example

Done: Gather the projective measurement statistics on a post-selected state. Had Python 3.13.1. Had to Install Python 3.11, since Qiskit’s packages (qiskit 2.0.1) are not yet officially compatible with Python 3.13.

QRAM: perceber como funciona para as assumptions — Emmanuel ????

PM: *Qiskit* uses little endian notation. Consider having a Summary of the Complete Algorithm/Protocol and Complexities for end-user applications as in <https://www.nature.com/articles/s41534-024-00883-0.pdf>

Explain how and why, in my experiments, adapting $w?v$, despite improvements, was worse than simply having a balanced trainign set. Also, say this is better for me to simply use H^* instead of some crazy differetn supeprpsotion in w because its a known operator, no added complexity hidden.

5.3 Real Network Data

The implementation and all data used in this work are openly available on GitHub.

5.3.1 Dataset Description

We construct our dataset from the OmniPath protein-protein interaction (PPI) network, accessed via the `OmniPathR` R package, which captures pairwise gene interactions at the protein level. Each node is annotated with its $-\log_{10}$ MutSig P -value [36], and nodes without MutSig values are removed. The resulting network contains 6 850 genes and 27 075 interactions, forming the candidate set for node classification. Ground-truth labels are obtained from the Cancer Gene Census (CGC) in the COSMIC database [1]. Tier 1 genes have documented cancer-relevant activity and mutations that promote oncogenic transformation, while Tier 2 genes show strong but less extensive evidence. Both Tier 1 and Tier 2 CGC genes are treated as positives, totaling 590 cancer genes ($\sim 8.6\%$ of the dataset). The remaining genes are unlabeled.

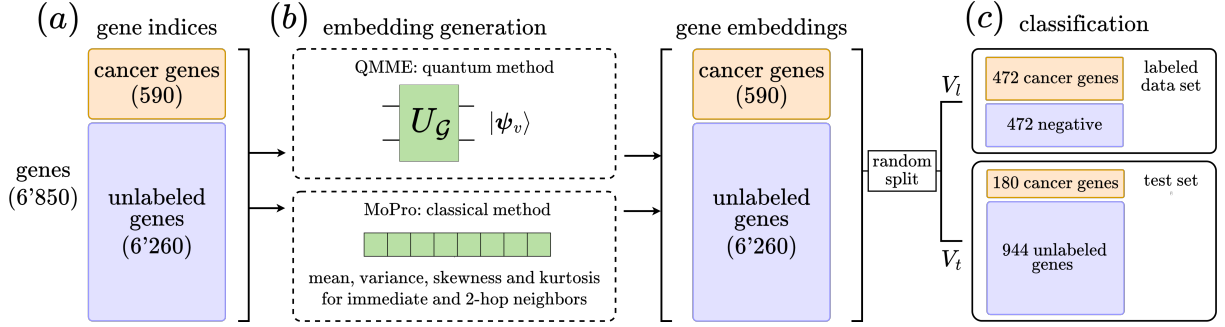


Figure 5.1: Experimental setup for QMME and classical embedding comparison and performance evaluation. (a) The dataset consists of 6 850 genes, of which 590 (in orange) are known cancer driver genes [1]. The remaining 6 260 genes have unknown labels, as they are not part of the CGC dataset. (b) Embeddings for each gene v are obtained via two methods: the Quantum Multi-order Moment Embedding (QMME) protocol, and the Moment Propagation (MoPro) embedding with hyperparameter $t = 1$ [2]. The QMME approach is implemented via algebraic computations on classical software. (c) In *each random split*, 80% of the positive genes are selected and combined with an equal number of unlabeled genes (assigned a negative class label) to form the labeled training set V_t . The remaining genes form the test set V_t , whose labels are used only for evaluation. For both embedding methods, kernel matrices are computed and the expectation values in eq. (4.9) are evaluated on all test genes. This process is repeated across 50 random splits, and performance metrics are averaged.

5.3.2 Performance Evaluation

In the constructed dataset, the number of positive samples (cancer driver genes) is much smaller than the pool of negative or unlabeled samples. If left uncorrected, this class imbalance biases the classifier toward predicting negatives. To mitigate this, the labeled training set V_t is constructed for each random split by selecting 80% of the known cancer driver genes and pairing them with an equal number of randomly sampled unlabeled genes, which are provisionally assigned to the negative class (Fig. 5.1c). This corresponds to a balanced set of 944 genes (472 positives and 472 negatives). Although some of the unlabeled genes may be undiscovered drivers, treating them as negatives provides a balanced ground truth to inform the classifier. The remaining genes form the disjoint test set V_t , and this process is repeated across 50 independent splits, with performance metrics averaged over all splits. Balancing is especially important in the quantum setting, as the first step of state preparation constructs a superposition over labeled nodes, and an imbalanced distribution would induce skewed amplitudes. For both QMME and classical Moment Propagation (MoPro) embeddings, we construct a kernel matrix K from the embedding vectors and use the same distance-based classification procedure to predict labels. This ensures performance differences are attributable solely to the embeddings rather than classifier choice, providing a fair comparison. Classification labels are predicted from expectation values as in eq. (4.11). Statistical significance of the differences between QMME and MoPro metrics is assessed via paired sample t-tests. Compared with the original MoPro pipeline [2], which applies multiple classifiers with optimized hyperparameters (logistic regression, random forest, SVM, gradient boosting) and propagation steps $t > 1$, ours uses only one-step moment embeddings and the same distance-based

Table 5.1: Cancer gene classification performance (%). Comparison of Quantum Multi-order Moment Embeddings (QMME) and Moment Propagation (MoPro) embeddings over 50 random splits. All metric differences are statistically significant except AUPRC.

Metric	QMME	MoPro	t-test
Accuracy	37.2	53.1	$t = -6.829, p = 0.00000122$
Precision	2.1	2.3	$t = -3.486, p = 0.00105$
Recall	65.9	53.3	$t = 5.341, p = 0.000238$
F1-Score	4.0	4.4	$t = -3.432, p = 0.00123$
AUPRC	1.9 ± 0.2	1.8 ± 0.2	—

classifier for both QMME and MoPro. This avoids unfair advantages from classifier choice or propagation, focusing solely on the quality of the embedding representations. Logistic regression models the posterior probability of a class by applying a sigmoid function to a linear combination of input features. Linear support vector machines determine a separating hyperplane that maximizes the margin between the two classes. Both logistic regression and linear SVMs involve the optimization of a weight vector and a bias term, and are therefore classified as linear models. In general, both classical and quantum approaches yield comparable outcomes, as their underlying principles do not differ substantially. Nevertheless, in detail, while absolute AUPRC values in Table 5.1 are low due to the rarity of positive samples in the test set, QMME embeddings achieve a slightly higher mean AUPRC (0.0191) compared with MoPro (0.0177), and also exhibit lower variance across splits (0.0015 vs 0.0016). This indicates that QMME provides a more consistent ranking of true positives across random splits. The QMME embeddings achieve substantially higher recall (65.9%) compared with MoPro (53.3%), corresponding to a 12.6 percentage point improvement. This indicates that QMME is more effective at identifying true cancer driver genes (and rare positives) despite the low precision and overall accuracy.

While the framework is described using quantum states and operations for conceptual alignment with quantum computing principles, all quantum procedures, including the preparation of embeddings via U_G , the application of the swap-test, and the evaluation of expectation values, are implemented as classical simulations. Specifically, the embedding vectors and their inner products are computed algebraically on a classical computer using parallelized C++ code, without executing any operations on physical quantum hardware. This approach allows practical evaluation of the protocol while preserving the theoretical structure of the QMME and swap-test framework.

5.4 Complexity Analysis

We analyze the complexity of the embedding circuit U_G in Appendix B.3. Let d' denote the precision (in qubits) used to encode both feature values and node degrees. The number of oracle calls required for a single application of U_G is constant. The Hadamard gates contribute a cost that scales as $\mathcal{O}(\log s)$

within the unitary operator $U_{\mathcal{G}}$, and as $\mathcal{O}(\log |V_i|)$ across the entire quantum pipeline. Overall, the dominant contribution to the total gate complexity arises from arithmetic operations that are part of oracle subroutines (see the Appendix B.3 for a detailed breakdown). Circuit depth is the number of sequential

Table 5.2: Complexity analysis for the QMME protocol. $U_{\mathcal{G}}$ complexity estimates for s -sparse and general graphs, in terms of gate count, circuit depth, qubit requirements, and query complexity (calls to the input model).

Metric	s -sparse	General
Gate complexity	$\mathcal{O}(\text{polylog } N + d')$	$\mathcal{O}(\text{polylog } N + d')$
Circuit depth	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Qubit count	$\mathcal{O}(\text{polylog } N + d')$	$\mathcal{O}(\text{polylog } N + d')$
Query complexity	$\mathcal{O}(\log^2 s)$	$\mathcal{O}(\log^2 N)$

layers of gates, i.e., the computation length, and measures time complexity. The number of qubits is the size of the quantum register, which measures space (memory) complexity. Thus, computation time is independent of network size, and space complexity is logarithmic on it.

6

Concluding Remarks

Contents

6.1	Conclusions	59
6.2	Limitations and Future Work	59

6.1 Conclusions

Quantum amplitude embeddings of neighbors' features moments can be used in other **types of problems**, as suggested in [43]. The reader might feel that the model used in this paper is in some respect artificial. In fact, this model is still far away from practically encountered situations. As was the one presented in [83].

In this work, we introduce an almost fully quantum pipeline for cancer driver gene prediction based on a swap-test classifier, requiring no classical optimization loops. All stages of encoding, embedding, and inference are performed within the quantum domain, requiring only minimal data access assumptions and limited classical post-processing. Our approach demonstrates logarithmic resource scaling together with solid performance on protein-protein interaction (PPI) network data. The proposed *Quantum Multi-order Moment Embedding* (QMME) is a general and extensible architecture for encoding local graph statistics into quantum states. While we focus on binary classification of genes as cancer drivers using PPI networks (with nodes representing genes and MutSig *P*-values as features), the QMME framework is broadly applicable to node-level classification tasks on feature-labeled graphs across domains [43]. Future directions include extending QMME to capture higher-order moments, supporting weighted graphs, enabling multi-class classification, and implementing resource-efficient variants suitable for near-term quantum hardware.

6.2 Limitations and Future Work

There are several important **caveats** to the QMME and QC protocols. First, (...). Finally, although the algorithm scales as (...), current estimates of the cost of the algorithm for practical problems are prohibitive or not, which underlines the importance of investigating further improvements. "In general, the promise of exponential speedups for linear systems should be tempered with the realization that they apply only to certain problems [39]. the execution of quantum algorithms requires quantum hardware that is not yet available."

PM: In [39], they pose 4 main challenges of quantum machine learning. Use them here or elsewhere!

Bibliography

- [1] J. G. Tate, S. Bamford, H. C. Jubb, Z. Sondka, D. M. Beare, N. Bindal, H. Boutselakis, C. G. Cole, C. Creatore, E. Dawson, P. Fish, B. Harsha, C. Hathaway, S. C. Jupe, C. Y. Kok, K. Noble, L. Ponting, C. C. Ramshaw, C. E. Rye, H. E. Speedy, R. Stefancsik, S. L. Thompson, S. Wang, S. Ward, P. J. Campbell, and S. A. Forbes, "Cosmic: the catalogue of somatic mutations in cancer," *Nucleic Acids Research*, vol. 47, no. D1, pp. D941–D947, 10 2018. [Online]. Available: <https://doi.org/10.1093/nar/gky1015>
- [2] A. C. Gumpinger, K. Lage, H. Horn, and K. Borgwardt, "Prediction of cancer driver genes through network-based moment propagation of mutation scores," *Bioinformatics*, vol. 36, no. Supplement_1, pp. i508–i515, 2020.
- [3] M. Soeken, M. Roetteler, N. Wiebe, and G. De Micheli, "Design automation and design space exploration for quantum computers," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*. Ieee, 2017, pp. 470–475.
- [4] D. K. Park, C. Blank, and F. Petruccione, "The theory of the quantum kernel-based binary classifier," *Physics Letters A*, vol. 384, no. 21, p. 126422, 2020.
- [5] L. A. Garraway and E. S. Lander, "Lessons from the cancer genome," *Cell*, vol. 153, no. 1, pp. 17–37, 2013.
- [6] B. Vogelstein, N. Papadopoulos, V. E. Velculescu, S. Zhou, L. A. Diaz Jr, and K. W. Kinzler, "Cancer genome landscapes," *science*, vol. 339, no. 6127, pp. 1546–1558, 2013.
- [7] G. M. Frampton, A. Fichtenholtz, G. A. Otto, K. Wang, S. R. Downing, J. He, M. Schnall-Levin, J. White, E. M. Sanford, P. An *et al.*, "Development and validation of a clinical cancer genomic profiling test based on massively parallel dna sequencing," *Nature biotechnology*, vol. 31, no. 11, pp. 1023–1031, 2013.
- [8] H. Horn, M. S. Lawrence, C. R. Chouinard, Y. Shrestha, J. X. Hu, E. Worstell, E. Shea, N. Ilic, E. Kim, A. Kamburov *et al.*, "Netsig: network-based discovery from cancer genomes," *Nature methods*, vol. 15, no. 1, pp. 61–66, 2018.

- [9] M. A. Reyna, M. D. Leiserson, and B. J. Raphael, "Hierarchical hotnet: identifying hierarchies of altered subnetworks," *Bioinformatics*, vol. 34, no. 17, pp. i972–i980, 2018.
- [10] I. A. Kovács, K. Luck, K. Spirohn, Y. Wang, C. Pollis, S. Schlabach, W. Bian, D.-K. Kim, N. Kishore, T. Hao *et al.*, "Network-based prediction of protein interactions," *Nature communications*, vol. 10, no. 1, p. 1240, 2019.
- [11] F. Cheng, I. A. Kovács, and A.-L. Barabási, "Network-based prediction of drug combinations," *Nature communications*, vol. 10, no. 1, p. 1197, 2019.
- [12] J. N. Weinstein, E. A. Collisson, G. B. Mills, K. R. Shaw, B. A. Ozenberger, K. Ellrott, I. Shmulevich, C. Sander, and J. M. Stuart, "The cancer genome atlas pan-cancer analysis project," *Nature genetics*, vol. 45, no. 10, pp. 1113–1120, 2013.
- [13] C. Kandoth, M. D. McLellan, F. Vandin, K. Ye, B. Niu, C. Lu, M. Xie, Q. Zhang, J. F. McMichael, M. A. Wyczalkowski *et al.*, "Mutational landscape and significance across 12 major cancer types," *Nature*, vol. 502, no. 7471, pp. 333–339, 2013.
- [14] N. D. Dees, Q. Zhang, C. Kandoth, M. C. Wendl, W. Schierding, D. C. Koboldt, T. B. Mooney, M. B. Callaway, D. Dooling, E. R. Mardis *et al.*, "Music: identifying mutational significance in cancer genomes," *Genome research*, vol. 22, no. 8, pp. 1589–1598, 2012.
- [15] D. C. Koboldt, Q. Zhang, D. E. Larson, D. Shen, M. D. McLellan, L. Lin, C. A. Miller, E. R. Mardis, L. Ding, and R. K. Wilson, "VarScan 2: somatic mutation and copy number alteration discovery in cancer by exome sequencing," *Genome research*, vol. 22, no. 3, pp. 568–576, 2012.
- [16] D. E. Larson, C. C. Harris, K. Chen, D. C. Koboldt, T. E. Abbott, D. J. Dooling, T. J. Ley, E. R. Mardis, R. K. Wilson, and L. Ding, "Somaticsniper: identification of somatic point mutations in whole genome sequencing data," *Bioinformatics*, vol. 28, no. 3, pp. 311–317, 2012.
- [17] A. Roth, J. Ding, R. Morin, A. Crisan, G. Ha, R. Giuliany, A. Bashashati, M. Hirst, G. Turashvili, A. Oloumi *et al.*, "Jointsvnmix: a probabilistic model for accurate detection of somatic mutations in normal/tumour paired next-generation sequencing data," *Bioinformatics*, vol. 28, no. 7, pp. 907–913, 2012.
- [18] K. Cibulskis, M. S. Lawrence, S. L. Carter, A. Sivachenko, D. Jaffe, C. Sougnez, S. Gabriel, M. Meyer, E. S. Lander, and G. Getz, "Sensitive detection of somatic point mutations in impure and heterogeneous cancer samples," *Nature biotechnology*, vol. 31, no. 3, pp. 213–219, 2013.
- [19] T. Sjöblom, S. Jones, L. D. Wood, D. W. Parsons, J. Lin, T. D. Barber, D. Mandelker, R. J. Leary, J. Ptak, N. Silliman *et al.*, "The consensus coding sequences of human breast and colorectal cancers," *science*, vol. 314, no. 5797, pp. 268–274, 2006.

- [20] L. D. Wood, D. W. Parsons, S. Jones, J. Lin, T. Sjoblom, R. J. Leary, D. Shen, S. M. Boca, T. Barber, J. Ptak *et al.*, "The genomic landscapes of human breast and colorectal cancers," *Science*, vol. 318, no. 5853, pp. 1108–1113, 2007.
- [21] S. Jones, X. Zhang, D. W. Parsons, J. C.-H. Lin, R. J. Leary, P. Angenendt, P. Mankoo, H. Carter, H. Kamiyama, A. Jimeno *et al.*, "Core signaling pathways in human pancreatic cancers revealed by global genomic analyses," *science*, vol. 321, no. 5897, pp. 1801–1806, 2008.
- [22] L. Ding, G. Getz, D. A. Wheeler, E. R. Mardis, M. D. McLellan, K. Cibulskis, C. Sougnez, H. Greulich, D. M. Muzny, M. B. Morgan *et al.*, "Somatic mutations affect key pathways in lung adenocarcinoma," *Nature*, vol. 455, no. 7216, pp. 1069–1075, 2008.
- [23] D. W. Parsons, S. Jones, X. Zhang, J. C.-H. Lin, R. J. Leary, P. Angenendt, P. Mankoo, H. Carter, I.-M. Siu, G. L. Gallia *et al.*, "An integrated genomic analysis of human glioblastoma multiforme," *science*, vol. 321, no. 5897, pp. 1807–1812, 2008.
- [24] C. G. A. R. N. T. source sites: Duke University Medical School McLendon Roger 1 Friedman Allan 2 Bigner Darrell 1, E. U. V. M. E. G. . . . B. D. J. . . M. M. G. . O. J. J. . . 5, H. F. H. M. T. . L. N. 8, M. A. C. C. A. K. . A. Y. W. . B. O. 11, U. of California San Francisco VandenBerg Scott 12 Berger Mitchel 13 Prados Michael 13 *et al.*, "Comprehensive genomic characterization defines human glioblastoma genes and core pathways," *Nature*, vol. 455, no. 7216, pp. 1061–1068, 2008.
- [25] C. G. A. R. Network *et al.*, "Integrated genomic analyses of ovarian carcinoma," *Nature*, vol. 474, no. 7353, p. 609, 2011.
- [26] B. . W. H. . H. M. S. C. L. . . P. P. J. . K. R. 13, G. data analysis: Baylor College of Medicine Creighton Chad J. 22 23 Donehower Lawrence A. 22 23 24 25, I. for Systems Biology Reynolds Sheila 31 Kreisberg Richard B. 31 Bernard Brady 31 Bressler Ryan 31 Erkkila Timo 32 Lin Jake 31 Thorsson Vestein 31 Zhang Wei 33 Shmulevich Ilya 31 *et al.*, "Comprehensive molecular portraits of human breast tumours," *Nature*, vol. 490, no. 7418, pp. 61–70, 2012.
- [27] M. J. Ellis, L. Ding, D. Shen, J. Luo, V. J. Suman, J. W. Wallis, B. A. Van Tine, J. Hoog, R. J. Goiffon, T. C. Goldstein *et al.*, "Whole-genome analysis informs breast cancer response to aromatase inhibition," *Nature*, vol. 486, no. 7403, pp. 353–360, 2012.
- [28] C. G. A. R. N. A. working group: Baylor College of Medicine Creighton Chad J. 1 2 Morgan Margaret 1 Gunaratne Preethi H. 1 3 Wheeler David A. 1 Gibbs Richard A. 1, B. C. A. G. R. A. . C. A. 4, B. I. B. R. . . C. K. 6, B. . W. H. S. S. . . 59, B. U. V. H.-T. W. F. . . R. B. J. 8, U. of Texas MD Anderson Cancer Center Verhaak Roel GW 9 Tamboli Pheroze 10 Torres-Garcia Wandaliz 9 Akbani Rehan 9 Weinstein John N. 9, M. S.-K. C. C. R. V. . H. J. J. . R. B. A. . A. H. A. . J. A. . C. G. . R. B. 13, N. C.

- I. R. C. J. . M. L. W. agibbs@bcm.edu 14 c, U. of California Santa Cruz Stuart Joshua M. 15, C. H. K. R. W. . University of North Carolina *et al.*, “Comprehensive molecular characterization of clear cell renal cell carcinoma,” *Nature*, vol. 499, no. 7456, pp. 43–49, 2013.
- [29] C. G. A. R. Network, “Genomic and epigenomic landscapes of adult de novo acute myeloid leukemia,” *New England Journal of Medicine*, vol. 368, no. 22, pp. 2059–2074, 2013.
- [30] D. A. Levine, C. G. A. R. N. G. sequencing centres: Broad Institute Getz Gad 1 Gabriel Stacey B. 1 Cibulskis Kristian 1 Lander Eric 1 Sivachenko Andrey 1 Sougnez Carrie 1 Lawrence Mike 1, W. U. in St Louis Kandoth Cyriac 2 Dooling David 2 Fulton Robert 2 Fulton Lucinda 2 Kalicki-Veizer Joelle 2 McLellan Michael D. 2 O’Laughlin Michelle 2 Schmidt Heather 2 Wilson Richard K. 2 Ye Kai 2 Ding Li 2 Mardis Elaine R. 2, U. of Southern California & Johns Hopkins Baylin Stephen B. 21 Bootwalla Moiz S. 22 Lai Phillip H. 22 Triche Jr Timothy J. 22 Van Den Berg David J. 22 Weisenberger Daniel J. 22 Laird Peter W. 22 Shen Hui 22, I. for Systems Biology Reynolds Sheila M. 23 Shmulevich Ilya 23 *et al.*, “Integrated genomic characterization of endometrial carcinoma,” *Nature*, vol. 497, no. 7447, pp. 67–73, 2013.
- [31] D. Hanahan and R. A. Weinberg, “The hallmarks of cancer,” *cell*, vol. 100, no. 1, pp. 57–70, 2000.
- [32] K. Lage, E. O. Karlberg, Z. M. Størling, P. I. Olason, A. G. Pedersen, O. Rigina, A. M. Hinsby, Z. Tümer, F. Pociot, N. Tommerup *et al.*, “A human phenome-interactome network of protein complexes implicated in genetic disorders,” *Nature biotechnology*, vol. 25, no. 3, pp. 309–316, 2007.
- [33] T. Li, R. Wernersson, R. B. Hansen, H. Horn, J. Mercer, G. Slodkiewicz, C. T. Workman, O. Rigina, K. Rapacki, H. H. Stærfeldt *et al.*, “A scored human protein–protein interaction network to catalyze genomic interpretation,” *Nature methods*, vol. 14, no. 1, pp. 61–64, 2017.
- [34] D. Szklarczyk, A. L. Gable, D. Lyon, A. Junge, S. Wyder, J. Huerta-Cepas, M. Simonovic, N. T. Doncheva, J. H. Morris, P. Bork *et al.*, “String v11: protein–protein association networks with increased coverage, supporting functional discovery in genome-wide experimental datasets,” *Nucleic acids research*, vol. 47, no. D1, pp. D607–D613, 2019.
- [35] J. G. Lohr, P. Stojanov, M. S. Lawrence, D. Auclair, B. Chapuy, C. Sougnez, P. Cruz-Gordillo, B. Knoechel, Y. W. Asmann, S. L. Slager *et al.*, “Discovery and prioritization of somatic mutations in diffuse large b-cell lymphoma (dlbcl) by whole-exome sequencing,” *Proceedings of the National Academy of Sciences*, vol. 109, no. 10, pp. 3879–3884, 2012.
- [36] M. S. Lawrence, P. Stojanov, C. H. Mermel, J. T. Robinson, L. A. Garraway, T. R. Golub, M. Meyer-son, S. B. Gabriel, E. S. Lander, and G. Getz, “Discovery and saturation analysis of cancer genes across 21 tumour types,” *Nature*, vol. 505, no. 7484, pp. 495–501, 2014.

- [37] P. Wittek, *Quantum machine learning: what quantum computing means to data mining*. Academic Press, 2014.
- [38] M. Schuld, I. Sinayskiy, and F. Petruccione, “An introduction to quantum machine learning,” *Contemporary Physics*, vol. 56, no. 2, pp. 172–185, 2015.
- [39] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.
- [40] J. P. Moutinho, A. Melo, B. Coutinho, I. A. Kovács, and Y. Omar, “Quantum link prediction in complex networks,” *Physical Review A*, vol. 107, no. 3, p. 032605, 2023.
- [41] D. Magano, J. Moutinho, and B. Coutinho, “On the quantum simulation of complex networks,” *SciPost Physics Core*, vol. 6, no. 3, p. 058, 2023.
- [42] J. P. Moutinho, D. Magano, and B. Coutinho, “On the complexity of quantum link prediction in complex networks,” *Scientific Reports*, vol. 14, no. 1, p. 1026, 2024.
- [43] W. Bi, L. Du, Q. Fu, Y. Wang, S. Han, and D. Zhang, “Mm-gnn: Mix-moment graph neural network towards modeling neighborhood feature distribution,” in *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*, 2023, pp. 132–140.
- [44] H. Zhao, L. Lewis, I. Kannan, Y. Quek, H.-Y. Huang, and M. C. Caro, “Learning quantum states and unitaries of bounded gate complexity,” *PRX Quantum*, vol. 5, no. 4, p. 040306, 2024.
- [45] Y. Wang and J. Liu, “A comprehensive review of quantum machine learning: from nisq to fault tolerance,” *Reports on Progress in Physics*, 2024.
- [46] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca, “Quantum algorithms revisited,” *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, vol. 454, no. 1969, pp. 339–354, 1998.
- [47] D. Anguita, S. Ridella, F. Riviello, and R. Zunino, “Quantum optimization for training support vector machines,” *Neural Networks*, vol. 16, no. 5-6, pp. 763–770, 2003.
- [48] S. Arora and B. Barak, *Computational complexity: a modern approach*. Cambridge University Press, 2009.
- [49] J. M. Martyn, Z. M. Rossi, A. K. Tan, and I. L. Chuang, “Grand unification of quantum algorithms,” *PRX quantum*, vol. 2, no. 4, p. 040203, 2021.
- [50] M. Möttönen, J. J. Vartiainen, V. Bergholm, and M. M. Salomaa, “Quantum circuits for general multiqubit gates,” *Physical review letters*, vol. 93, no. 13, p. 130502, 2004.

- [51] R. A. Servedio and S. J. Gortler, “Equivalences and separations between quantum and classical learnability,” *SIAM Journal on Computing*, vol. 33, no. 5, pp. 1067–1092, 2004.
- [52] D. P. DiVincenzo, “Two-bit gates are universal for quantum computation,” *Physical Review A*, vol. 51, no. 2, p. 1015, 1995.
- [53] D. Camps, L. Lin, R. Van Beeumen, and C. Yang, “Explicit quantum circuits for block encodings of certain sparse matrices,” *SIAM Journal on Matrix Analysis and Applications*, vol. 45, no. 1, pp. 801–827, 2024.
- [54] Y. Tong, D. An, N. Wiebe, and L. Lin, “Fast inversion, preconditioned quantum linear system solvers, fast green’s-function computation, and fast evaluation of matrix functions,” *Physical Review A*, vol. 104, no. 3, p. 032422, 2021.
- [55] G. Verdon, T. McCourt, E. Luzhnica, V. Singh, S. Leichenauer, and J. Hidary, “Quantum graph neural networks,” *arXiv preprint arXiv:1909.12264*, 2019.
- [56] M. Schuld and N. Killoran, “Quantum machine learning in feature hilbert spaces,” *Physical review letters*, vol. 122, no. 4, p. 040504, 2019, see Supplemental Material at <https://link.aps.org/supplemental/10.1103/PhysRevLett.122.040504> for additional details.
- [57] T. Hofmann, B. Schölkopf, and A. J. Smola, “Kernel methods in machine learning,” 2008.
- [58] C. Blank, D. K. Park, J.-K. K. Rhee, and F. Petruccione, “Quantum classifier with tailored quantum kernel,” *npj Quantum Information*, vol. 6, no. 1, p. 41, 2020, see Supplemental Material at https://static-content.springer.com/esm/art%3A10.1038%2Fs41534-020-0272-6/MediaObjects/41534_2020_272_MOESM1_ESM.pdf for additional details.
- [59] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, “Circuit-centric quantum classifiers,” *Physical Review A*, vol. 101, no. 3, p. 032308, 2020.
- [60] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, “The theory of variational hybrid quantum-classical algorithms,” *New Journal of Physics*, vol. 18, no. 2, p. 023023, 2016.
- [61] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi *et al.*, “Pennylane: Automatic differentiation of hybrid quantum-classical computations,” *arXiv preprint arXiv:1811.04968*, 2018.
- [62] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, “Quantum circuit learning,” *Physical Review A*, vol. 98, no. 3, p. 032309, 2018.

- [63] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, “Supervised learning with quantum-enhanced feature spaces,” *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.
- [64] M. J. Bremner, A. Montanaro, and D. J. Shepherd, “Average-case complexity versus approximate simulation of commuting quantum computations,” *Physical review letters*, vol. 117, no. 8, p. 080501, 2016.
- [65] S. Lloyd, M. Mohseni, and P. Rebentrost, “Quantum algorithms for supervised and unsupervised machine learning,” *arXiv preprint arXiv:1307.0411*, 2013.
- [66] M. Schuld, M. Fingerhuth, and F. Petruccione, “Implementing a distance-based classifier with a quantum interference circuit,” *Europhysics Letters*, vol. 119, no. 6, p. 60002, 2017.
- [67] C. Miquel, J. P. Paz, M. Saraceno, E. Knill, R. Laflamme, and C. Negrevergne, “Interpretation of tomography and spectroscopy as dual forms of quantum computation,” *Nature*, vol. 418, no. 6893, pp. 59–62, 2002.
- [68] H. Buhrman, R. Cleve, J. Watrous, and R. De Wolf, “Quantum fingerprinting,” *Physical review letters*, vol. 87, no. 16, p. 167902, 2001.
- [69] C. T. Hann, C.-L. Zou, Y. Zhang, Y. Chu, R. J. Schoelkopf, S. M. Girvin, and L. Jiang, “Hardware-efficient quantum random access memory with hybrid quantum acoustic systems,” *Physical review letters*, vol. 123, no. 25, p. 250501, 2019.
- [70] P. Rebentrost, M. Mohseni, and S. Lloyd, “Quantum support vector machine for big data classification,” *Physical review letters*, vol. 113, no. 13, p. 130503, 2014.
- [71] J. Shao, *Mathematical statistics*. Springer, 1999.
- [72] H. Kwon, H. Lee, and J. Bae, “Feature map for quantum data in classification,” in *2024 International Conference on Quantum Communications, Networking, and Computing (QCNC)*. IEEE, 2024, pp. 41–48.
- [73] O. Goldreich and D. Ron, “Property testing in bounded degree graphs,” in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, 1997, pp. 406–415.
- [74] O. Goldreich, *Introduction to property testing*. Cambridge University Press, 2017.
- [75] S. Ben-David, A. M. Childs, A. Gilyén, W. Kretschmer, S. Podder, and D. Wang, “Symmetries, graph properties, and quantum speedups,” *SIAM Journal on Computing*, vol. 53, no. 6, pp. FOCS20–368, 2024.

- [76] L. Lin, "Lecture notes on quantum algorithms for scientific computation," *arXiv preprint arXiv:2201.08309*, 2022.
- [77] P. Kuklinski and B. Rempfer, "S-fable and ls-fable: Fast approximate block-encoding algorithms for unstructured sparse matrices," *arXiv preprint arXiv:2401.04234*, 2024.
- [78] A. Zaman, H. J. Morrell, and H. Y. Wong, "A step-by-step hhl algorithm walkthrough to enhance understanding of critical quantum computing concepts," *IEEE Access*, vol. 11, pp. 77 117–77 131, 2023.
- [79] G. Sergioli, R. Giuntini, and H. Freytes, "A new quantum approach to binary classification," *PloS one*, vol. 14, no. 5, p. e0216224, 2019.
- [80] L. Cincio, Y. Subaşı, A. T. Sornborger, and P. J. Coles, "Learning the quantum algorithm for state overlap," *New Journal of Physics*, vol. 20, no. 11, p. 113022, 2018.
- [81] D. C. Luna-Márquez, R. Pinto-Elías, T. Pérez-Becerra, A. Magadán-Salazar, N. González-Franco, and J. A. Fuentes-Pacheco, "knn classification using hausdorff metric with quantum-based euclidean distance," *Quantum Machine Intelligence*, vol. 7, no. 1, p. 54, 2025.
- [82] P. Erdős and P. Kelly, "The minimal regular graph containing a given graph," *American Mathematical Monthly*, vol. 70, no. 10, pp. 1074–1075, 1963.
- [83] M. Sasaki and A. Carlini, "Quantum learning and universal quantum matching machine," *Physical Review A*, vol. 66, no. 2, p. 022303, 2002.
- [84] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," *Physical review letters*, vol. 103, no. 15, p. 150502, 2009.



Code of Project

Notice: mathematics uses Formulas, well rendered in documents produced with LaTeX.

This inline MATLAB code `for i=1:3, disp('cool'); end;` uses the `\mcode{}` command.¹

Listing A.1: Matlab Function

```
1 for i = 1:3
2     if i >= 5 && a ~= b           % literate programming replacement
3         disp('cool');             % comment with some  $\pi x^2$ 
4     end
5     [:,ind] = max(vec);
6     x_last = x(1,end) - 1;
7     v(end);
8     ylabel('Voltage ( $\mu V$ )');
9 end
```

¹MATLAB Works also in footnotes: `for i=1:3, disp('cool'); end;`

Listing A.2: function.m

```
1 % Copyright 2010 The MathWorks, Inc.
2 function ObjTrack(position)
3 % #codegen
4 % First, setup the figure
5 numPts = 300;           % Process and plot 300 samples
6 figure;hold;grid;       % Prepare plot window
7 % Main loop
8 for idx = 1: numPts
9     z = position(:,idx); % Get the input data
10    y = kalmanfilter(z);  % Call Kalman filter to estimate the position
11    plot_trajectory(z,y); % Plot the results
12 end
13 hold;
14 end % of the function
```

Listing A.3: PYTHON Code

```
1 class TelegramRequestHandler(object):
2     def handle(self):
3         addr = self.client_address[0]           # Client IP-adress
4         telegram = self.request.recv(1024)      # Recieve telegram
5         print "From: %s, Received: %s" % (addr, telegram)
6         return
```

B

Drafts and A Large Table

B.1 Classification Unitary

In this representation, $\hat{y}_v^{\text{class}} = 1$ indicates node v is classified as cancerous (i.e., a known cancer gene), and $\hat{y}_v^{\text{class}} = 0$ indicates a unknown-status node (likely healthy). Thus, the classifier learns to map input nodes to their corresponding health status based on the quantum measurement outcome.

Given N training data points of the form $\{(\vec{x}_v, y_v) : \vec{x}_v \in \mathbb{R}^8, y_v \in \{0, 1\}\}_{v \in [N]}$, where y_v indicates the class label of $\vec{x}_j$, the task reduces to finding the unitary W as described in ???. The goal is to map the input state $|\psi_{\text{in}}\rangle$, which encodes the amplitude-encoded training data, to the output state $|\psi_{\text{labels}}\rangle$, which uniformly encodes the known labels. This corresponds to solving the unitary ???, a constrained linear mapping problem equation. Conceptually, this resembles solving a regression or classification problem via matrix inversion, but under the constraint that W must be unitary.

To solve for W , we require labeled training data. The classification state $|\psi_{\text{QCP}}\rangle$ must be encoded from known labels. Once W is determined, it can be applied to unseen nodes for classification.

Directly calculating W without training. Derive the eight-parameter W analytically, e.g., using matrix inversion. More as a deterministic solution to a linear system than an iterative training model.

Kronecker product, not outer product!

Solving ?? reduces to finding a unitary $W \in \mathbb{C}^{8 \times 8}$, such that, for each fixed v , it maps the 8-dimensional feature vector $\vec{u}$ to one of the canonical base vectors, or not.

For more than 8 examples, I can't simultaneously do this unless all $\vec{u}_v$ lie in a subspace that allows to define such a mapping. A unitary exists if the set $\{\vec{u}\}$ is orthonormal (or can be orthogonalized), and each maps to a different label vector. A unitary can't map arbitrary feature vectors to arbitrary labels unless there's some structure (orthogonality, low rank, etc).

Imagine we are doing a linear separation of 2 points with 2 labels in a 2D-space. 2 points is exactly what we need to build a linear system of 2 equations with 2 incognitos $ax + b = 0$. The closed-form solution won't be scalable for large datasets (>8 nodes).

B.1.1 HHL-inspired linear regression/classification (Amplitude-Encoding the Class)

The state generated by the QMMEP is given by

$$|\psi_{\text{QMME}}\rangle = \sum_{v=1}^N \sum_{j=1}^8 u_{v,j} |v\rangle |j\rangle = \sum_{v=1}^N |v\rangle |\vec{u}_v\rangle, \quad (\text{B.1})$$

which is the input for the QCP, with $|\vec{u}_v\rangle = \sum_{j=1}^8 u_{v,j} |j\rangle$.

PM: Linear regression model (as in HHL). Then, threshold for linear classification (as in QSVMs for Big Data). For a general form for a hyperplane in the mapped space, see the nice paper [47], also about Quantum computing for SVM training.

The Linear System Problem (LSP) can be represented as $A\vec{x} = \vec{b}$ [84]. In our case,

$$A = (\vec{u}_1, \dots, \vec{u}_v, \dots, \vec{u}_N)^T, \quad (\text{B.2})$$

with $A \in \mathbb{R}^{N \times 8}$, $\vec{b} \in \mathbb{C}^8$. Since our A is not Hermitian, it can, in theory, be converted to a Hermitian matrix as $\tilde{A} = \begin{bmatrix} 0 & A \\ A^\dagger & 0 \end{bmatrix}$, with $A \in \mathbb{R}^{(N+8) \times (N+8)}$, $\vec{b} \in \mathbb{C}^{(N+8)}$.

The unitary W shall act on the feature space and compute the corresponding label. In practice, W may be implemented as a parameterized quantum circuit consisting of gates conditioned on the feature register. Specifically, for a given input vector $\vec{u}_v$, one may encode the label prediction into a dedicated classification qubit via a rotation gate, such as $R_y(\theta(\vec{u}_v)) |0\rangle = \cos\left(\frac{\theta(\vec{u}_v)}{2}\right) |0\rangle + \sin\left(\frac{\theta(\vec{u}_v)}{2}\right) |1\rangle = |\hat{y}_v^{\text{class}}\rangle$. To satisfy $W |\psi_{\text{QMME}}\rangle = |\psi_{\text{QCP}}\rangle$ as stated in ??, consider $W = I^{\otimes N} \otimes I^{\otimes 8} \otimes R_y(\theta(\vec{u}_v))$, where the final rotation acts on the classification qubit, conditioned (implicitly or explicitly) on the features. In practice, $R_y(\theta(\vec{u}_v))$ could be decomposed into a set of feature-controlled rotations (parameterized gates), depending on the encoding scheme. If the features were basis-encoded rather than amplitude-encoded, the classification logic could be implemented via controlled unitaries (e.g., controlled- X or controlled

Table B.1: Example table

Benchmark: ANN	#Layers (1)	#Nets (2)	#Nodes* (3) = 8 · (1) · (2)	Critical path (4) = 4 · (1)	Latency (T_{iter}) (5)
A1	3–1501	1	24–12008	12–6004	4
A2	501	1	4008	2004	2–2000
A3	10	2–1024	160–81920	40	60 [†]
A4	10	50	4000	40	80–1200
Benchmark: FFT	FFT size [‡] (1)	#Inputs (2) = 2 ⁽¹⁾	#Nodes* (3) = 10 · (1) · (2)	Critical path (4) = 4 · (1)	Latency (T_{iter}) (5)
F1	1–10	2–1024	20–102400	4–40	6–60 [†]
F2	5	32	1600	20	40 – 1500
Benchmark: Random networks	#Types (1)	#Nodes (2)	#Networks (3)	Critical path (4)	Latency (T_{iter}) (5)
R1	3	10–2000	500	variable	(4)
R2	3	50	500	variable	(4) × [1; ⋯ ; 20]

* Excluding constant nodes.

[†] Value kept proportional to the critical path: (5) = (4) · 1.5.

[‡] A size of x corresponds to a 2^x point FFT.

Values in bold indicate the parameter being varied.

rotations) acting on the label qubit, with controls based on the feature basis states.

Matrix A is encoded as the Hamiltonian of a unitary gate, and we expect the controlled- U operation to be implemented by Hamiltonian simulation [78, 84]. However, in the latter, the matrix is derived for U and then mapped to the Controlled- U gate used in IBM-Q directly.

As Table B.1 shows, the data can be inserted from a file, in the case of a somehow complex structure. Notice the Table footnotes.

And now an example (Table B.3) of a table that extends to more than one page. Notice the repetition of the Caption (with indication that is continued) and of the Header, as well as the continuation text at the bottom.

Table B.2: Comparison between today's and target Architectures of Telcos

Today		Target	
Rigid	Each evolutionary requirement involves development of multiple components, interfaces, platforms, etc.	Flexible	It is possible to modify or add new functionalities rapidly.
Slow	Development of a new application takes months or years.	Fast	Development of a new application takes weeks instead of months or years.
Closed	Limited integration with external environments.	Open	It is simple to integrate internal, applications with external entities.
Complex	Heterogeneous technologies, obsolescence, lack of standards, high redundancy.	Standardised	Use of homogeneous architectural models.
Expensive	High Capex (for new service development) and high Opex (to ensure running of IT).	Cost-Effective	Capex and Opex are optimised.

Table B.3: Example of a very long table spreading in several pages

Time (s)	Triple chosen	Other feasible triples
0	(1, 11, 13725)	(1, 12, 10980), (1, 13, 8235), (2, 2, 0), (3, 1, 0)
2745	(1, 12, 10980)	(1, 13, 8235), (2, 2, 0), (2, 3, 0), (3, 1, 0)
5490	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
8235	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)

An example of a large Table that autofits the size to the page margins is illustrated in Table B.4. Please notice the text size that is shrunk in order for the table to adjust to the page:

Table B.4: Sample Table.

URL	First Time Visit	Last Time Visit	URL Counts	Value	Reference
https://web.facebook.com/	1521241972	1522351859	177	56640	[facebook-2021]

B.2 Quantum Node Embedding Step-by-step Derivation

U_r denotes the operator U acting on register r , with identity on all other registers; $C - U_r$ the controlled application of U_r , with the control qubit in register c being in state $|1\rangle$; and $C^{(0)} - U_r$ indicates control on

the $|0\rangle$ state of c .

$|v, 0^m\rangle$ where $m = 2 \log N + 8$, omitting work qubits,

$$\begin{aligned}
& \xrightarrow{I_{n-s} \otimes H_{l_1}^{\otimes s}} \frac{1}{\sqrt{s}} |v, 0^3\rangle \sum_{\ell_1 \in [s]} |\ell_1, 0^{(n+5)}\rangle \\
& \xrightarrow{(O_L)_{v, l_1}} \frac{1}{\sqrt{s}} |v, 0^3\rangle \sum_{\ell_1 \in [s]} |r(v, \ell_1), 0^{(n+5)}\rangle \\
& \xrightarrow{H_c} \frac{1}{\sqrt{2s}} |v\rangle \sum_{c \in [2]} \sum_{\ell_1 \in [s]} |c, 0^2, r(v, \ell_1), 0^{(n+5)}\rangle \\
& \xrightarrow{I_{n-s} \otimes C \cdot H_{\ell_2}^{\otimes s}} \frac{1}{\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \left(|0_c, 0^2, r(v, \ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |1_c, 0^2, r(v, \ell_1), \ell_2, 0^5\rangle \right) \\
& \xrightarrow{C \cdot (O_L)_{\ell_1, \ell_2}} \frac{1}{\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \left(|0_c, 0^2, r(v, \ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |1_c, 0^2, r(v, \ell_1), r(\mathcal{N}_v^1(\ell_1), \ell_2), 0^5\rangle \right) \\
& \xrightarrow{H_i^{\otimes 2}} \frac{1}{2^{3/2} \sqrt{s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{i \in [4]} \left(|0_c, i, r(v, \ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |1_c, i, r(v, \ell_1), r(\mathcal{N}_v^1(\ell_1), \ell_2), 0^5\rangle \right) \\
& \xrightarrow{C^{(0)} \cdot (O_X)_{i, a, l_1}} \frac{1}{2^{3/2} \sqrt{s}} |v\rangle \sum_{\ell_1, i} \left((x_{r(v, \ell_1)}^i |0_c, i, r(v, \ell_1), 0^{(n+4)}\rangle + |\perp\rangle) |0\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |1_c, i, r(v, \ell_1), r(\mathcal{N}_v^1(\ell_1), \ell_2), 0^5\rangle \right) \\
& \xrightarrow{C \cdot (O_X)_{i, a, \ell_2}} \frac{|v\rangle}{2^{3/2} \sqrt{s}} \sum_{\ell_1, i} \left(x_{r(v, \ell_1)}^i |0_c, i, r(v, \ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} x_{r(\mathcal{N}_v^1(\ell_1), \ell_2)}^i |1_c, i, r(v, \ell_1), r(\mathcal{N}_v^1(\ell_1), \ell_2), 0^5\rangle \right) + |v\rangle |\perp\rangle \\
& \xrightarrow{C \cdot (O_L^T)_{\ell_1, \ell_2}} \frac{1}{2^{3/2} \sqrt{s}} |v\rangle \sum_{\ell_1, i} \left(x_{r(v, \ell_1)}^i |0_c, i, r(v, \ell_1), 0^{(n+5)}\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} x_{r(\mathcal{N}_v^1(\ell_1), \ell_2)}^i |1_c, i, r(v, \ell_1), \ell_2, 0^5\rangle \right) + |v\rangle |\perp\rangle |0\rangle \\
& \xrightarrow{I_{n-s} \otimes C \cdot H_{\ell_2}^{\otimes s}} \frac{1}{2^{3/2} \sqrt{s}} |v\rangle \sum_{\ell_1, i} \left(x_{r(v, \ell_1)}^i |0_c, i, r(v, \ell_1)\rangle + \frac{1}{\sqrt{s}} \sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |1_c, i, r(v, \ell_1)\rangle \right) |0^{(n+5)}\rangle + |v\rangle |\perp\rangle |0\rangle \\
& \xrightarrow{(O_L^T)_{v, l_1}} \frac{1}{2^{3/2} \sqrt{s}} |v\rangle \sum_{\ell_1, i} \left(x_{r(v, \ell_1)}^i |0_c, i, \ell_1\rangle + \frac{1}{s} \sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |1_c, i, \ell_1\rangle \right) |0^{(n+5)}\rangle + |v\rangle |\perp\rangle |0\rangle \\
& \xrightarrow{I_{n-s} \otimes H_{l_1}^{\otimes s}} \frac{1}{2^{3/2} s} |v\rangle \sum_{i \in [4]} \left(\sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |0_c\rangle + \frac{1}{s} \sum_{x'' \in \mathcal{X}_v^2} x''^i |1_c\rangle \right) |i, 0^{(2n+5)}\rangle + |v\rangle |\perp\rangle |0\rangle \\
& \xrightarrow{(O_{K-1})_{v, k}} \frac{1}{2^{3/2} s} |v\rangle \sum_{i \in [4]} \left(\sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |0_c\rangle + \frac{1}{s} \sum_{x'' \in \mathcal{X}_v^2} x''^i |1_c\rangle \right) \frac{1}{k_v^c} |i, 0^{(2n+5)}\rangle + |v\rangle |\perp\rangle \\
& = |v\rangle \left(\sum_{i, c} \frac{1}{2^{3/2} s^c} \sum_{x_u \in \mathcal{X}_v^c} \frac{x_u^i}{k_v^c} |c, i, 0^{(2n+5)}\rangle + |\perp\rangle \right) \\
& = |v\rangle \left(\sum_{i, c} \frac{1}{2^{3/2} s^c} \varphi(\mathcal{X}_v^c) |c, i, 0^{(2n+5)}\rangle + |\perp\rangle \right) \\
& = |v\rangle \left(\sum_{j \in [8]} c_j m_{v, j} |j, 0^{(2n+5)}\rangle + |\perp\rangle \right) \quad \text{with } c \text{ defined as in eq. (3.34).}
\end{aligned}$$

Quantum Circuit for Amplitude Encoding $\mu(\mathcal{X}_v^1)$

Proof. We apply the first four gate sets to the source state:

$$|v, 0^m\rangle \xrightarrow{I_{n-s} \otimes H_{t_1}^{\otimes s}} \xrightarrow{(O_L)_{v, t_1}} \xrightarrow{H_c} \frac{1}{\sqrt{2s}} |v\rangle \sum_{c \in [2]} \sum_{\ell_1 \in [s]} |c, 0^2, r(v, \ell_1), 0^{(n+5)}\rangle \quad (\text{B.3})$$

$$\xrightarrow{O_x} \frac{1}{\sqrt{Ns}} |0\rangle \sum_{v \in [N]} \sum_{\ell \in [s]} \left(x_j |0\rangle + \sqrt{1 - |x_j|^2} |1\rangle \right) |j\rangle |v\rangle \quad (\text{B.4})$$

where $j = r(v, \ell)$. We then apply O_{K-1} , $I_{n-s} \otimes H^{\otimes s}$, and O_L to the target state:

$$|0\rangle_{a_k} |0\rangle_{a_x} |0^n\rangle_{t_1} |v\rangle \xrightarrow{O_{K-1}} \xrightarrow{I_{n-s} \otimes H^{\otimes s}} \xrightarrow{O_L} \quad (\text{B.5})$$

$$\frac{1}{\sqrt{s}} \sum_{\ell' \in [s]} \left(\frac{1}{k_v} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v} \right|^2} |1\rangle \right) |0\rangle |r(v, \ell') = i\rangle |v\rangle. \quad (\text{B.6})$$

Then, the inner product, for the simplest case, where $c, i = 1$, gives

$$\langle v, 0, 0 | \langle 0 |^{\otimes (m-3)} U_G |v\rangle |0\rangle^{\otimes m} = \frac{1}{2^{3/2} s k_v} \sum_{l, l'} x_j \delta_{r(v, l), r(v, l')} \quad (\text{B.7})$$

$$= \frac{1}{2^{3/2} s k_v} \sum_l x_{r(v, l)} \quad (\text{B.8})$$

$$= \frac{1}{2^{3/2} s} \sum_{u \in \mathcal{N}_1^v} \frac{x_u}{|\mathcal{N}_1^v|} \quad (\text{B.9})$$

$$= \frac{1}{2^{3/2} s} \sum_{x_u \in \mathcal{X}_v^1} \frac{x_u}{k_v^1} \quad (\text{B.10})$$

$$= \frac{\varphi_1(\mathcal{X}_v^1)}{2^{3/2} s}. \quad (\text{B.11})$$

□

B.3 Complexity Analysis in Detail

Each quantum circuit can be characterized by (1) Gate Count, the total number of qubits involved; (2) Gate Depth, the length of the longest sequence of gates applied sequentially; (3) Number of Work Qubits, which are used to store intermediate results; and (4) Query Complexity, the number of oracle calls required. We can construct a quantum circuit to implement oracles O_L and $\tilde{O}_X$ with $\mathcal{O}(\text{polylog } N + d')$ elementary gates [54]. The rotation operators are simply controlled versions of single-qubit $R_y(\theta)$ rotations, with the angle computed from the d' -qubits in the control register. Using simple fixed-point arithmetic, such operators scale as $\mathcal{O}(d')$.

Table B.5: Quantum gate complexity and queries for various sources.

Operator	Description	Gate Complexity	Qubit Count	Query Complexity
H	elementary gate	1	1	–
O_L	input model	$\mathcal{O}(\text{polylog } N + d')$	$\mathcal{O}(\log N)$	$\mathcal{O}(1)$
$\tilde{O}_X$	input model	$\mathcal{O}(\text{polylog } N + d')$	$\mathcal{O}(\log N + d')$	$\mathcal{O}(1)$
$\tilde{O}_K$	derived from O_L	$\mathcal{O}(\text{polylog } N + d')$	$\mathcal{O}(\log N + d')$	$\mathcal{O}(\log^2 s)$
F_X	type of R_y	$\mathcal{O}(d')$	$\mathcal{O}(d')$	–
$F_{K^{-1}}$	type of R_y (inverse)	$\mathcal{O}(d')$	$\mathcal{O}(d')$	–
O_X	$1\tilde{O}_X \cdot 5\mathbf{c}\text{-}F_X \cdot 1\tilde{O}_X^T$	$\mathcal{O}(\text{polylog } N + d')$	$\mathcal{O}(\log N + d')$	–
$O_{K^{-1}}$	$1\tilde{O}_K \cdot 1F_{K^{-1}} \cdot 1\tilde{O}_K^T$	$\mathcal{O}(\text{polylog } N + d')$	$\mathcal{O}(\log N + d')$	$\mathcal{O}(\log^2 s)$

